GreenLit Go – Movie Success Prediction using Data Analytics

A PROJECT REPORT

Mini Project (01CE0609)

Submitted by

Rushit Trambadia
92410103114

Vandit Doshi
92410103050

BACHELOR OF TECHNOLOGY
in
Computer Engineering

Faculty of Engineering & Technology
Marwadi University, Rajkot

April, 2026

Mini Project (01CE0609)

Department of Computer Engineering

Faculty of Engineering & Technology

Marwadi University

A.Y. 2025-26

CERTIFICATE

This is to certify that the project report submitted along with the project entitled GreenLit Go – Movie Success Prediction using Data Analytics has been carried out by Rushit Trambadia (92410103114), Vandit Doshi (92410103050) under my guidance in partial fulfillment for the degree of Bachelor of Technology in Computer Engineering, 6th Semester of Marwadi University, Rajkot during the academic year 2025-26.

Prof. Priyanka Mangi	Prof. (Dr.) Rachit Adhvaryu
Assistant Professor	Head of the Department
Department of Computer Engineering	Department of Computer Science Engineering

Mini Project (01CE0609)

Department of Computer Engineering

Faculty of Engineering & Technology

Marwadi University

A.Y. 2025-26

DECLARATION

We hereby declare that the **Mini Project (01CE0609)** report submitted along with the Project entitled GreenLit Go – Movie Success Prediction using Data Analytics submitted in partial fulfilment for the degree of Bachelor of Technology in Computer Engineering to Marwadi University, Rajkot, is a bonafide record of original project work carried out by me / us at Marwadi University under the supervision of Prof. Priyanka Mangi and that no part of this report has been directly copied from any students' reports or taken from any other source, without providing due reference.

	Name of the Student	Sign of Student
1	_____	_____
2	_____	_____

Acknowledgement

We would like to express our deepest gratitude to all those who contributed to the successful completion of our project, “GreenLit GO - Movie Success Prediction Using Data Analytics”. This endeavour would not have been possible without the unwavering support, guidance, and encouragement of the following individuals and institutions.

We extend our sincere thanks to Prof. (Dr.) Krunal Vaghela, Professor, Dean & Head of the Computer Engineering Department, and Prof. (Dr.) Rachit Adhvaryu, Head of the Computer Science Department at Marwadi University, for their invaluable guidance, critical feedback, and continuous motivation throughout the project. Their expertise and patience helped us navigate challenges and refine our solution.

We are equally grateful to our project guide, Prof. Priyanka Mangi, for her technical insights, brainstorming sessions, and relentless support during the development phase. Her practical advice bridged the gap between theoretical concepts and real-world implementation.

We acknowledge the Faculty of Engineering & Technology, Marwadi University, for providing us with the necessary infrastructure, resources, and a conducive environment to conduct our research and development. The university’s emphasis on innovation and problem-solving inspired us to address complex industry challenges through technology.

This project is a testament to teamwork and collaboration. We thank each team member, Rushit and Vandit, for their dedication and constructive debates that shaped the project’s outcome. Every contribution, whether in machine learning, frontend design, documentation, or debugging, was pivotal to our success.

Team Members

Rushit Trambadia

Vandit Doshi

April 2026

Marwadi University, Rajkot

Abstract

Greenlighting a film is a high-stakes financial gamble, yet studios continue to rely heavily on intuition when deciding which projects to fund. ***GreenLit GO*** *was built to change that — a practical, web-based platform that brings data-driven decision-making into the heart of the theatrical greenlighting process.*

The platform draws on a dataset of over 2,180 films spanning Hollywood and Bollywood, aggregated from sources like TMDB, OMDB, YouTube, and Box Office Mojo. From this raw data, 46 engineered features were derived to reflect the real-world factors that shape box office performance — including cast and director popularity, trailer-driven online buzz, seasonal timing, and relative production budgets.

At its core, GreenLit GO uses a Stacking Ensemble model that combines Random Forest, XGBoost, and LightGBM, with hyperparameters tuned through Optuna's Bayesian optimization. The result is an R^2 *score of 0.75, classifying films across five outcome tiers from 'Flop' to 'Blockbuster.' Crucially, the model doesn't operate as a black box, SHAP-based explainability breaks down the contribution of each factor behind every prediction.*

The full application is built on a React/Vite frontend, a Node.js backend, and a Python Flask microservice handling the ML pipeline, all connected to MongoDB Atlas. Users can explore industry trends through interactive dashboards, run "what-if" simulations by tweaking production variables, and upload raw screenplay PDFs for AI-powered narrative feedback via the Gemini API.

Together, these capabilities position GreenLit GO as a comprehensive analytical toolkit for modern film development.

List of Figures

Fig 1.1 Gantt Chart..... 12
Fig 3.1 System Architecture 23
Fig 3.2 ML Workflow Diagram 24
Fig 3.3 Physical ER Diagram 28
Fig 3.4 DFD Diagram Level 0 29
Fig 3.5 DFD Diagram Level 1 30
Fig 3.6 Use Case Diagram 31
Fig 3.7 Class Diagram 32
Fig 3.8 Sequence Diagram 33
Fig 3.9 Activity Diagram 33
Fig 3.10 State Transition Diagram 34
Fig 3.11 Prediction Form Input 36
Fig 3.12 Prediction Results 37
Fig 4.1 Home Page – Hero …..... 46
Fig 4.2 Home Page – Capabalities 46
Fig 4.3 Home Page – Stats & Footer 47
Fig 4.4 Released Movies Page – Hero 47
Fig 4.5 Released Movies Page – Movies Library 48
Fig 4.6 Released Movies Page – Discover (User Recommendation System) 49
Fig 4.7 Released Movies Page – Discover (By Mood) 49
Fig 4.8 Released Movies Page – Discover (By Genre) 49
Fig 4.9 Released Movies Page – Discover (By Similar Movies) 50
Fig 4.10 Released Movies Page – Discover (By Talent) 50
Fig 4.11 Released Movies Page – Discover (By Top Rated) 50
Fig 4.12 Released Movies Page – Discover (Suprise Me) 50
Fig 4.13 Insights Page – Hero 51
Fig 4.14 Insights Page – Insights 51
Fig 4.15 Insights Page – Insights 52
Fig 4.16 Insights Page – Insights 52
Fig 4.17 Upcoming Movies Page – Hero 53

Fig 4.18 Upcoming Movies Page – Prediction Form and Prediction Output 53
Fig 4.19 Upcoming Movies Page – What If Simulation 54
Fig 4.20 Upcoming Movies Page - Optimal Release 55
Fig 4.21 About Us Page – Hero 56
Fig 4.22 About Us Page - About Info 56
Fig 4.23 About Us Page - Technology Used 56
Fig 4.24 About Us Page - What We Offer 57
Fig 4.25 About Us Page - Builders and CTA Section 57

List of Tables

Table 1.1 Technology Stack 4
Table 1.2 Study of Academic Reasearch 7
Table 1.3 Gap Analysis 9
Table 1.4 Sprint Table 10
Table 1.5 Project Effort Table 11
Table 1.6 Roles and Responsibiltes 11
Table 2.1 Functional Requirements 14
Table 2.2 Non Functional Requirements 16
Table 2.3 Modules of the System 19
Table 2.4 Hardware Selection 20
Table 2.5 Software Selection 21
Table 3.1 Prediction Input Form Fields 35
Table 3.2 Prediction Output Report 37
Table 3.3 Access Control / Security 38
Table 4.1 Development Enviorment 39
Table 4.2 Runtime Configuration 39
Table 4.3 Enviorment Variables 40
Table 4.4 Frontend Pages 43
Table 4.5 Revenue Prediction Model Performance 43
Table 4.6 Classification Model Performance 44
Table 4.7 Top Features by Importance 44
Table 4.8 Feature Engineering Test Cases 58
Table 4.9 Prediction Logic Test Cases 59
Table 4.10 API Endpoint Test Cases 60
Table 4.11 Integration Test Cases 61
Table 4.12 System Test Cases 62
Table 4.13 ML Validation Tests 62
Table 5.1 Project Outcomes 69

Abbreviations

KNN	**K-Nearest Neighbors**
IMDb	**Internet Movie Database**
ML	**Machine Learning**
API	**Application Programming Interface**
TMDB	**The Movie Database**
OMDB	**Open Movie Database**
AI	**Artificial Intelligence**
CSS	**Cascading Stylesheet**
UI/UX	**User Interface/User Experience.**
ROI	**Return on Investment**
MPAA	**Motion Picture Association of America**
SHAP	**SHapley Additive exPlanations**
BOM	**Box Office Mojo**
FR	**Functional Requirements**
NFR	**Non Functional Requirements**
JSON	**JavaScript Object Notation**
DFD	**Data Flow Diagram**
UML	**Unified Modeling Language**

Table of Contents

Acknowledgement i
Abstract ii
List of Figures iii
List of Tables v
List of Abbreviations vi
Table of Contents vii
Chapter 1 Introduction 1
1.1 Project Summary 1
1.2 Purpose 2
1.3 Objective 2
1.4 Scope 3
1.5 Technology and Literature Review 4
1.6 Project Planning and Scheduling 10
1.6.1 Project Development Approach and Justification 10
1.6.2 Project Effort and Time, Cost Estimation 11
1.6.3 Roles and Responsibilities 11
1.6.4 Group Dependencies 11
1.7 Project Scheduling 12
Chapter 2 System Analysis 13
2.1 Study of Current System 13
2.2 Problem and Weaknesses of Current System 13
2.3 Requirements of New System 14
2.4 System Feasibility 16
2.4.1 Does the system contribute to the overall objectives of the organization? 16
2.4.2 Can the system be implemented using the current technology and with the given cost and schedule constraints 16
2.4.3 Can the system be integrated with other systems which are already in place? . 17
2.5 Activity / Process in New System / Proposed System 17
2.6 Features of New System / Proposed System 18

2.7 List Main Modules / Components / Processes / Techniques of NewSystem/ Proposed System …. 19
2.8 Selection of Hardware / Software / Algorithms / Methodology / Techniques / …. 20
Chapter 3 System Design 23
3.1 System Design & Methodology.... 23
3.2 Database Design / Data Structure Design / Process Design / Structure Design/ ERDiagram / DFD /UML etc 25
3.3 Input / Output and Interface Design (If applicable) 34
3.3.1 State Transition Diagram (optional) 34
3.3.2 Samples of Forms, Reports and Interface 35
3.3.3 Access Control / Mechanism / Security (If applicable) 38
Chapter 4 Implementation & Testing 39
4.1 Implementation Platform / Environment 39
4.2 Process / Program / Technology / Modules Specification(s) 40
4.3 Finding / Results / Outcomes 43
4.4 Result Analysis / Comparison / Deliberations 45
4.5 Snapshots 46
4.6 Testing Plan / Strategy 58
4.7 Test Results and Analysis 58
4.7.1 Test Cases 58
Chapter 5 Conclusion and Future Enhancements 64
5.1 Overall Analysis of Project Viabilities 64
5.2 Problem Encountered and Possible Solutions 65
5.3 Summary of Project work 66
5.4 Limitations 67
5.5 Future Enhancement 68
5.6 Project Outcomes and discussion 69
References 71

CHAPTER 1

INTRODUCTION TO PROJECT AND PROJECT MANAGEMENT

1.1 Project Summary

GreenLit Go is a web-based movie success prediction platform that helps filmmakers, producers, and film enthusiasts estimate how well a movie might perform at the box office — before it even releases. The name is a nod to the industry term "greenlight," which is the moment a studio officially approves a movie for production.

At its core, the system pulls together data from more than 4,000 films across five film industries (Hollywood, Bollywood, Tollywood, Kollywood, and Mollywood), engineers 46 numerical features from that raw data, and feeds them into a stacking ensemble of machine learning models — Random Forest, XGBoost, LightGBM, and KNN — to predict revenue and classify movies into categories like Blockbuster, Super Hit, Hit, Average, or Flop.

What sets this project apart from similar academic work is how we handle data leakage — a common problem where models train on information that would not exist for an unreleased movie. Features like opening weekend gross, IMDb ratings, and critic scores are available for historical films in the training set and help the model learn richer revenue patterns. But for upcoming movies, these values obviously do not exist yet. Rather than simply dropping this useful training signal, we built a smart imputation system that estimates these missing post-release values from correlated pre-release signals like budget tier, talent popularity, and trailer engagement. This leakage-aware approach gives our model an honest R^2 of about 0.75, reflecting genuine forecasting ability for movies that have not released yet. these are commonly used in published research, but they defeat the whole purpose of "predicting" anything, since you cannot know a movie's opening weekend before it opens. By stripping out these leaky signals, our model gives an honest R^2 of about 0.75, which reflects genuine forecasting ability.

The application itself is built as a three-tier system: a React frontend for the user interface, a Node.js/Express backend for API routing and database access, and a Flask microservice that handles all the ML inference. MongoDB Atlas stores the movie data in the cloud.

1.2 Purpose

The purpose of GreenLit Go is to bring a data-driven perspective into the movie greenlighting process. Right now, the decision to invest tens of millions (or even hundreds of millions) of dollars into a film project is largely based on intuition, past experience, and industry relationships. While these human factors are valuable, they leave a lot of money on the table — or worse, drive it into projects that were always going to struggle commercially.

This platform does not aim to replace human judgement. Rather, it gives decision-makers one more tool in their toolkit: a quantitative estimate of a movie's commercial potential based on historically observed patterns. If someone is considering whether to greenlight a $150 million action film with a particular director and cast lineup, they can punch those details into the system and get a grounded projection, along with a clear breakdown of which factors are pushing the prediction up or down.

For students and researchers, the platform also serves as a practical demonstration of how machine learning pipelines are built, deployed, and integrated into real web applications — complete with data engineering, model training, hyperparameter tuning, explainability, and full-stack frontend/backend development.

1.3 Objective

The specific objectives of this project are:

1. To aggregate movie data from multiple external sources — TMDB, OMDB, YouTube Data API, and Box Office Mojo — and consolidate it into a unified MongoDB database covering 4,000+ films across five industries.
2. To engineer a set of 46 pre-release features from raw movie metadata, including inflation-adjusted budgets, cyclic release timing, talent popularity metrics, trailer engagement scores, and production value ratios.

3. To design a leakage-aware training pipeline that identifies post-release features (opening gross, audience ratings, critic scores) and implements intelligent imputation for these values when predicting unreleased movies, ensuring the model remains genuinely useful for pre-release forecasting.
4. To build a stacking ensemble model combining Random Forest, XGBoost, LightGBM, and KNN regressors under a Ridge meta-learner, with hyperparameters optimized through Optuna's Bayesian search, achieving an R^2 of at least 0.70 on the held-out test set.
5. To classify each movie prediction into one of five success categories (Blockbuster, Super Hit, Hit, Average, Flop) using an ROI-based classification scheme with industry-specific revenue thresholds.
6. To provide per-prediction explainability so users can see which input factors (budget, star power, release timing, genre, etc.) contributed most to the model's output.
7. To deliver a production-ready web application with a movie browser, analytics dashboard (15+ chart types), interactive prediction sandbox, and AI-powered script analysis module.

1.4 Scope

What the system can do:

- Predict the approximate box office revenue of a movie using only information available before release (budget, cast, director, genre, release month, industry, trailer engagement).
- Classify the prediction into five outcome categories based on projected return on investment.
- Provide a visual breakdown of feature contributions for each individual prediction.
- Let users browse and filter 4,000+ released movies by industry, genre, year, and success category.
- Display analytics insights through interactive charts covering revenue trends, genre performance, seasonal patterns, and talent comparisons.
- Analyze a movie script or synopsis through the Gemini AI API and return structured commercial potential scores.

- Handle five film industries with calibrated revenue thresholds so that a Bollywood film is not evaluated against Hollywood benchmarks.

What the system cannot do:

- It cannot predict with certainty. Even the best models leave roughly 25% of variance unexplained. Unexpected events (pandemics, controversies, viral marketing) are beyond the model's reach.
- It does not track real-time box office performance or streaming numbers.
- It does not analyze live social media sentiment (Twitter/X, Reddit, Instagram).
- It does not provide geographic revenue breakdowns (domestic vs. international vs. China).
- It does not support user accounts, authentication, or personalized recommendation at this time.
- It does not account for marketing spend, which is a significant driver of box office performance but is rarely disclosed publicly.

1.5 Technology and Literature Review

Table 1.1 Technology Stack

Layer	Technology	Purpose
Frontend	React 18 + Vite	Single-page application with component-based UI
Styling	Vanilla CSS + CSS Custom Properties	Glassmorphic dark-mode design system
Backend	Node.js + Express 4	RESTful API routing, database access, proxy to ML service
ML Service	Python 3.10 + Flask 3	Model loading, feature preparation, inference, explainability

Database	MongoDB Atlas 7	Cloud-hosted NoSQL document storage for 4,000+ movie records
ML Models	Scikit-learn, XGBoost, LightGBM	Stacking ensemble for regression and classification
Tuning	Optuna	Bayesian hyperparameter optimization
Explainability	Custom feature importance (SHAP-inspired)	Per-prediction contribution analysis
AI Integration	Google Gemini API	Script analysis for commercial potential scoring
Data Sources	TMDB, OMDB, YouTube Data API, Box Office Mojo	Raw movie metadata, social metrics, box office data

Literature Review

To establish the context, feasibility, and specific requirements for the GreenLit Go platform, a comprehensive literature survey was conducted. The survey was divided into two distinct domains: an analysis of existing commercial platforms currently operating in the film industry, and a review of modern academic research regarding machine learning models in box-office prediction.

Survey of Commercial Platforms

Several enterprise-level platforms currently attempt to solve the problem of cinema revenue forecasting. A review of these platforms highlights the existing industry approach:

1. **Cinelytic (cinelytic.com) — Industry Leader**
    - **Background:** Founded in Los Angeles (2014). Key clients include Warner Bros, STX Entertainment, and Sony Pictures.
    - **Functionality:** An AI-driven content intelligence platform for forecasting film revenue, casting analytics, and release strategy optimization. Provides

probabilistic revenue forecasts, return on investment (ROI) modeling, and distribution recommendations.

- **Claims:** Reports ~85% predictive accuracy at the pre-greenlight stage and 99% for US box office up to 1 year in advance. Uses a subscription-based enterprise business model.
- **Limitations:** Extremely expensive and limited to enterprise clients. It is a closed-source "black-box" model offering no explainability, and is primarily focused on the Hollywood ecosystem.

2. **ScriptBook (scriptbook.io) — Script-Level AI**
    - **Background:** Founded in Belgium (2015).
    - **Functionality:** AI analysis of screenplays using Natural Language Processing (NLP) to predict commercial and critical success. Features 360° script analysis, MPAA rating prediction, and gender equality metrics (e.g., the Bechdel Test).
    - **Claims:** 87% accuracy in predicting financial performance purely from script text. Charges roughly $5,000 per analysis.
    - **Limitations:** Requires a completed screenplay, which restricts early pre-production analysis. It does not factor in dynamic market trends or trailer analytics, and is prohibitively expensive for independent creators.
3. **Vault AI (vault-ai.com)**
    - **Background:** Israel-based predictive content intelligence platform for global media streamers, broadcasters, and studios.
    - **Functionality:** Provides audience prediction, content performance forecasting, and marketing strategy optimization across 60+ countries.
    - **Limitations:** Exclusively an enterprise-only service with no public access or transparent pricing. It utilizes a proprietary black-box architecture lacking SHAP explainability.
4. **The Numbers (the-numbers.com) & Box Office Mojo**
    - **Functionality:** Provide historical box-office tracking, basic statistical analysis, historical comparisons, and industry reports.

- **Limitations:** These tools are purely descriptive. They offer no predictive machine learning capabilities, no future forecasting, and no classification mechanisms for upcoming movies.

Survey of Academic Research

In addition to commercial tracking, recent academic approaches state-of-the-art architectures for solving box office prediction problems:

Table 1.2 Survey of Academic Research

Sr.	Paper Title	Authors / Source	Year	Algorithms Used	Dataset Details	Key Results / Limitations
1	Movie Revenue Prediction using Machine Learning Models	V. Udandarao, P. Gupta (arXiv: 2405.11651)	2024	Linear Regression, Decision Tree, Random Forest, Bagging, XGBoost, Gradient Boosting	IMDB + TMDB (movie name, budget, genre, cast, director, runtime, MPAA rating)	Demonstrated promising accuracy strictly dependent on hyperparameter tuning and robust cross-validation.
2	Long-Range Movie Box Office Prediction Based on Machine Learning	J. Xu (Highlights in Sci., Engg. & Tech., Vol. 92)	2024	Random Forest, Back Propagation Neural Network, Linear Regression	TMDB (popularity, budget, genre, runtime, release date, language)	Achieved 73% prediction precision specifically calibrated for long-range temporal forecasting.
3	Box Office Prediction Model Based on	S. Tang (PLoS ONE, peer-reviewed)	2024	Optimized XGBoost compared against DNN,	Film marketing & box office dataset	Achieved highly optimized metrics:

	Optimized XGBoost Algorithm			LightGBM, Random Forest, GBDT, CatBoost	(2,500+ records)	Accuracy (0.90), F1 Score (0.90), Stability (0.98), Robustness (0.97).
4	Movie Box Office Prediction With Self-Supervised and Visually Grounded Pretraining	Q. Chao, E. Kim, B. Li (IEEE ICME 2023 / arXiv)	2023	BERT + Self-supervised visual pretraining on movie posters	Massive corpus of 35,794 movies	Yielded a 14.5% relative performance gain over standard finetuned BERT by incorporating visual grounding.
5	Predicting Movie Box Office Based on ML, DL, and Statistical Methods	BUCEA (Applied & Comp. Engg., CONF-MLA 2024)	2024	Comprehensive comparison spanning ML, Deep Learning, and Statistical approaches	Standardized box office dataset	Validated the superior capability of ensemble machine learning models over pure statistical inferences.
6	A Machine Learning Approach to Predict Movie Revenue	Q. I. Mahmud, et al. (J. of Computer Science, Vol 16)	2020	Logistic Regression, SVM, Multi-Layer Perceptron (MLP)	3,200 movies with 35 specific pre-release parameters	Scored 85.31% "one class away" accuracy while specifically classifying

	Based on Pre-Released Movie Metadata					outcomes from 'Flop' to 'Blockbuster'.

Identification of Need (Gap Analysis)

Based on the survey of existing commercial solutions and academic frameworks, critical gaps were identified in current offerings. **GreenLit Go** was directly conceptualized and architected to resolve these specific shortcomings, effectively democratizing access to high-tier analytics

Table 1.3 Gap Analysis

Identified Gap in Existing Solutions	**The GreenLit Go Solution**
Prohibitively Expensive (Often costing $5,000+ per analysis minimum)	Free and Open-Source accessible platform tailored for independent filmmakers and students.
Black-Box Predictions (Providing data with no explanation of why)	Integrates SHAP (SHapley Additive exPlanations) to provide transparent, feature-level explainability.
Western/Hollywood-Only Focus	Trained across 5 diverse industries, specifically bridging the gap by including Indian cinema (Bollywood, Tollywood, Kollywood, Mollywood).
Isolated Datasets (Relying only on script OR only on market data)	Utilizes a Multi-Source Pipeline combining TMDB (metadata), OMDB (ratings), YouTube (trailer virality), and BOM.

1.6 Project Planning and Scheduling

1.6.1 Project Development Approach and Justification

We followed an Agile-Incremental development approach with five iterative sprints. This was chosen over a traditional Waterfall model for a practical reason: the ML component of this project required multiple rounds of experimentation. We could not fully define the feature set, model architecture, or accuracy targets upfront because these emerged through trial and error.

For example, our data leakage audit in Sprint 4 led us to build the imputation pipeline and required re-engineering major parts of the prediction workflow.

Under a rigid Waterfall plan, this kind of mid-project pivot would have been extremely disruptive. Under Agile, it was simply the next sprint.

Table 1.4 Sprint Table

Sprint	Focus	Duration
Sprint 1	Data collection (TMDB, OMDB, YouTube, BOM), MongoDB setup	2 weeks
Sprint 2	Feature engineering, initial model training, baseline metrics	2 weeks
Sprint 3	Frontend development (Home, Movies, Insights, Upcoming pages)	3 weeks
Sprint 4	Data leakage audit, imputation pipeline, Optuna tuning, retraining	2 weeks
Sprint 5	UI polish, script analysis, industry-aware classification, testing	2 weeks

1.6.2 Project Effort and Time, Cost Estimation

Table 1.5 Project Effort Table

Resource	Estimated Effort	Actual Effort
Data Collection & Cleaning	25 hours	30 hours
Feature Engineering & ML Pipeline	40 hours	50 hours
Frontend Development	35 hours	40 hours
Backend API Development	20 hours	22 hours
Testing & Debugging	15 hours	20 hours
Documentation	10 hours	12 hours
Total	**145 hours**	**174 hours**

Cost: The project was built entirely using free and open-source tools. MongoDB Atlas free tier, TMDB/OMDB free API keys, YouTube Data API free quota, and Gemini API free tier. No hardware purchases were required — development was done on existing personal laptops. The effective monetary cost was zero.

1.6.3 Roles and Responsibilities

Table 1.6 Roles and Responsibiltes

Team Member	Role	Responsibilities
Rushit Trambadia	Full-Stack Developer & ML Engineer	Data pipeline, ML training, backend APIs, frontend, integration
Vandit Doshi	Frontend Developer	Frontend, UI & UX

1.6.4 Group Dependencies

The frontend team depended on the backend API being functional before they could integrate real data into the UI components. Similarly, the prediction sandbox page could not be completed until the ML service was trained, serialized, and serving predictions through

its Flask endpoints. The Insights dashboard depended on MongoDB aggregation pipelines being defined in the backend routes.

To manage these dependencies, we used stub data and mock API responses during early frontend development, which allowed UI work to proceed in parallel with backend and ML development.

1.7 Project Scheduling

Gantt Chart:

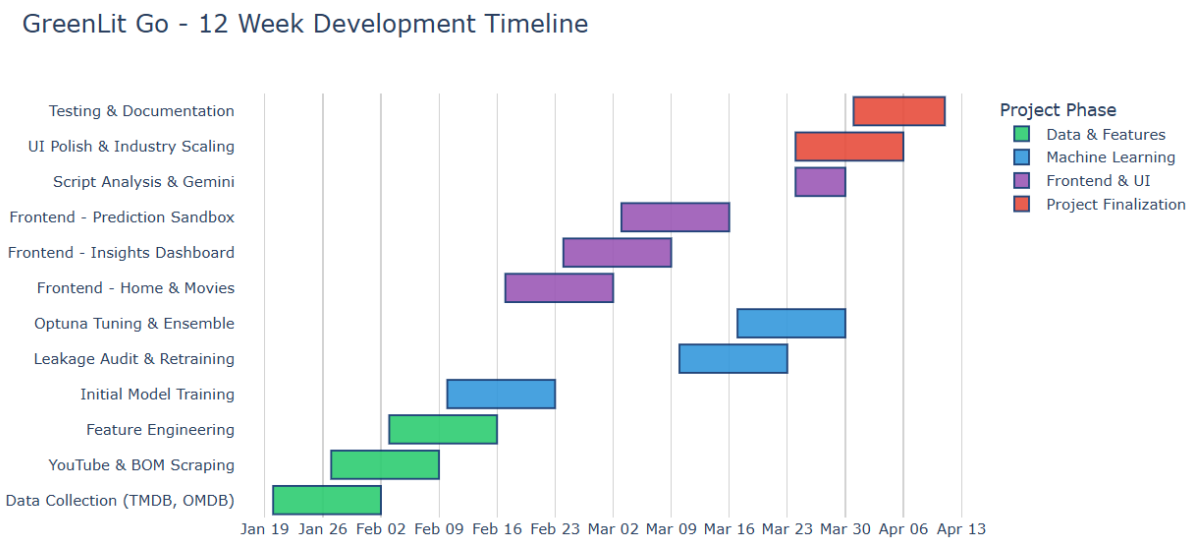

Figure 1.1 Gantt Chart

CHAPTER 2

SYSTEM ANALYSIS

2.1 Study of Current System

Before building GreenLit Go, we looked at what already exists in the space of movie performance prediction. The existing landscape falls into three broad categories:

Academic Research Models: There is a solid body of published papers on movie box office prediction. However, these models typically exist as Jupyter notebooks or research scripts. They are not deployed anywhere, have no user interface, and are generally inaccessible to anyone outside the research team. More critically, a large majority of them include post-release features like opening weekend gross and critic scores in their training data, which makes their reported accuracy figures misleading.

Studio-Internal Forecasting Tools: Major studios like Disney, Warner Bros., and Universal reportedly have in-house forecasting teams and proprietary tools. These are not publicly available for obvious competitive reasons. Independent filmmakers and smaller production houses have no access to these resources.

Box Office Tracking Websites: Platforms like Box Office Mojo, The Numbers, and Nash Information Services provide excellent historical data — past revenues, weekend rankings, theatre counts — but they do not offer predictive capabilities. You can look up how much "Avengers: Endgame" made, but you cannot ask "how much will my upcoming action movie make?"

2.2 Problem and Weaknesses of Current System

After studying the existing options, we identified the following specific weaknesses:

1. **Data Leakage is Pervasive:** The single biggest problem. When a model relies on the opening weekend gross to "predict" total revenue for an unreleased film, it is not really

predicting anything — because that data simply does not exist yet. Most published work includes these features without any mechanism to handle their absence for new movies. The inflated accuracy numbers this produces give a false sense of what is actually achievable in a real forecasting scenario.

2. **No Interactive Tool for Non-Technical Users:** A producer or filmmaker cannot open a Jupyter notebook and run Python code. They need a visual interface where they can type in movie details and see results.
3. **Single-Industry Focus:** Almost every existing model focuses exclusively on Hollywood. A Bollywood producer looking for guidance on their ₹200 crore project will find nothing relevant.
4. **No Explainability:** Most models output a number (predicted revenue) or a label (hit/flop) without any explanation. Users have no way to understand what is driving the prediction, which makes it difficult to trust or act on.
5. **No Inflation Adjustment:** Comparing a 1995 movie's $200M revenue directly against a 2024 movie's $200M revenue is misleading because the dollar has lost significant purchasing power over 29 years. Most existing models ignore this.
6. **Static and Outdated Datasets:** Many published models train on datasets that were collected once and never updated, meaning they do not reflect current market realities (e.g., post-pandemic theatrical trends).

2.3 Requirements of New System

Based on the weaknesses identified above, the requirements for GreenLit Go are:

Functional Requirements

Table 2.1 Functional Requirements

Req ID	Requirement	Priority
FR-01	System must predict box office revenue for unreleased movies by imputing any unavailable	Must Have

	post-release features from pre-release signals	
FR-02	System must classify predictions into 5 success categories (Blockbuster, Super Hit, Hit, Average, Flop)	Must Have
FR-03	System must provide per-prediction feature importance breakdown	Must Have
FR-04	System must support 5 film industries with tailored revenue thresholds	Must Have
FR-05	System must normalize all financial values to 2024 dollars for training	Must Have
FR-06	System must allow users to browse and filter released movies	Should Have
FR-07	System must provide analytics dashboards with 15+ chart types	Should Have
FR-08	System must support talent search (find director/actors via API)	Should Have
FR-09	System must offer AI-based script analysis	Nice to Have
FR-10	System must handle edge cases gracefully (zero budget, missing cast)	Must Have

Non-Functional Requirements

Table 2.2 Non Functional Requirements

Req ID	Requirement	Priority
NFR-01	Prediction response time under 5 seconds	Must Have
NFR-02	Responsive design — works on desktop, tablet, and mobile	Should Have
NFR-03	Clean error messages when ML service is unavailable	Must Have
NFR-04	Models must load on startup without requiring retraining	Must Have

2.4 System Feasibility

2.4.1 Does the system contribute to the overall objectives of the organization?

Yes. For a production studio or an independent filmmaker, the system provides a quantitative second opinion before committing significant capital to a film project. It does not replace creative judgment — nobody should abandon a great script just because the model predicts average revenue — but it adds a data layer to the decision-making process. For an academic institution, the project demonstrates a complete end-to-end ML system pipeline, from data collection to deployment, which is a valuable educational outcome.

2.4.2 Can the system be implemented using the current technology and within the given cost and schedule constraints?

Yes. Every technology used in this project is freely available and well-documented:

- React, Node.js, Express, Flask, and MongoDB are all open-source with large community support.
- Scikit-learn, XGBoost, LightGBM, and Optuna are free Python libraries.
- TMDB and OMDB offer free API tiers. YouTube Data API provides a daily free quota.

- MongoDB Atlas has a free-tier cluster that comfortably holds our 4,000-movie dataset.
- Development was done on personal laptops with no additional hardware costs.

The entire project was completed within one semester using these zero-cost resources.

2.4.3 Can the system be integrated with other systems which are already in place?

Yes. The system's architecture is designed around RESTful APIs, which makes integration straightforward:

- The Flask ML service exposes /api/predict and /api/explain endpoints that any external system can call with a simple HTTP POST request.
- The Express backend can be extended with new routes to serve additional frontend clients (e.g., a mobile app).
- The MongoDB database can be accessed by any application with the connection URI.
- The prediction output is returned as standard JSON, which is universally parseable.

2.5 Activity / Process in New System

The core workflow in GreenLit Go follows this sequence:

1. **User Inputs Movie Details:** The user fills out a form with the movie's title, budget, genres, release month, industry, and optionally searches for specific director and cast members through the TMDB talent search.
2. **Frontend Sends Request:** The React frontend sends the form data as a JSON payload to the Express backend via an Axios POST request.
3. **Backend Proxies to ML Service:** The Express server forwards the prediction request to the Flask ML microservice running on port 5001.
4. **Feature Preparation:** The ML service takes the raw input and constructs a 46-feature vector. For upcoming (unreleased) movies, missing values like vote counts and popularity scores are imputed from correlated known signals (budget tier, talent score, sequel status, trailer buzz).

5. **Model Inference:** The feature vector is scaled using the pre-fitted StandardScaler, then passed through the stacking ensemble regressor (for revenue) and the stacking ensemble classifier (for success category).
6. **Post-Processing:** The raw model output (log-scale revenue) is converted back to nominal dollars with industry-specific scaling applied. The success category is checked against ROI-based sanity rules and overridden if necessary.
7. **Feature Importance Calculation:** For this specific prediction, the system computes which features contributed most by multiplying each feature's scaled value by the model's global importance weight.
8. **Response Delivery:** The complete prediction payload — predicted revenue, success category, ROI estimate, predicted audience rating, confidence level, and top contributing features — is sent back through the Express server to the React frontend.
9. **Display:** The frontend renders the results in animated gauge cards with a feature importance bar chart.

2.6 Features of New System

1. **Leakage-Aware Prediction Engine:** Trains on all available data for historical films to learn rich revenue patterns, but intelligently imputes post-release features (opening gross, critic scores, vote counts) for upcoming movies using correlated pre-release signals, ensuring genuine forecasting utility.
2. **Multi-Industry Intelligence:** Supports five film industries with calibrated revenue thresholds and scaling factors.
3. **46-Feature Engineering Pipeline:** Includes inflation normalization, cyclic temporal encoding, talent synergy metrics, and trailer engagement ratios.
4. **Stacking Ensemble with Optuna Tuning:** Four base models combined under a Ridge meta-learner, each individually tuned via Bayesian optimization.
5. **Per-Prediction Explainability:** Every prediction comes with a ranked list of contributing factors, personalized to the specific movie's input profile.
6. **Interactive Prediction Sandbox:** Real-time talent search, budget sliders, genre selection, and instant prediction results with animated visualizations.

7. **Analytics Dashboard:** Over 15 chart types including histograms, scatter plots, bar charts, radar charts, and heat maps covering revenue trends, genre performance, and seasonal patterns.
8. **Released Movie Browser:** Paginated, filterable library of 4,000+ movies with poster cards, ROI badges, and detailed movie pages.
9. **AI Script Analysis:** Gemini API integration evaluates screenplay synopses for commercial potential, pacing, genre fit, and audience appeal.
10. **Responsive Design:** Functions on desktop, tablet, and mobile screens.

2.7 List Main Modules / Components of New System

Table 2.3 Modules of the System

Module	Technology	Description
Data Collection Module	Python (requests, BeautifulSoup)	Four collectors (TMDB, OMDB, YouTube, BOM) gather raw data and import into MongoDB
Feature Engineering Module	Python (NumPy, Pandas)	Transforms raw movie documents into a structured 46-column feature matrix
Model Training Module	Python (Scikit-learn, XGBoost, LightGBM, Optuna)	Tunes hyperparameters, trains base models, assembles stacking ensembles, evaluates via cross-validation
Prediction Service Module	Python (Flask)	Loads serialized models, prepares features, runs inference, computes feature importance, returns JSON
Backend API Module	Node.js (Express)	RESTful endpoints for movies, talents, predictions,

		trends, scripts; proxies ML requests
Frontend UI Module	React (Vite)	Seven pages — Home, Released Movies, Movie Detail, Insights, Upcoming Prediction, Script Analysis, About
Database Module	MongoDB Atlas	Cloud-hosted document store with flexible schema for multi-source movie data
Script Analysis Module	Google Gemini API	AI-driven evaluation of movie scripts for commercial viability

2.8 Selection of Hardware / Software and Justification

Hardware Selection

Table 2.4 Hardware Selection

Component	Specification	Justification
Processor	Intel i5 8th Gen or higher	Sufficient for model inference; training is a one-time batch process
RAM	8 GB minimum (16 GB recommended)	Model training with cross-validation can consume 6-8 GB at peak
Storage	2 GB free	Model pickle files (~500 MB combined), dataset, and application code
GPU	Not required	All models are tree-based (RF, XGB, LGBM); GPU acceleration not needed

Network	Stable broadband	Required for MongoDB Atlas connection and external API calls

Software Selection and Justification

Table 2.5 Software Selection

Software	Justification
React 18	Component-based architecture makes complex UI (dashboards, interactive forms) manageable. Vite bundler provides fast hot-reloading during development.
Node.js + Express	Non-blocking I/O handles concurrent API requests efficiently. Large ecosystem of npm packages.
Flask	Lightweight Python web framework, ideal for serving ML models. No overhead of Django's ORM or admin panel, which we do not need.
MongoDB Atlas	NoSQL document model naturally fits movie data, which has variable nested structures (cast arrays, YouTube metrics objects, BOM data). No rigid schema migrations needed when adding new data sources.
Scikit-learn StackingRegressor	Proven implementation of the stacking ensemble pattern. Handles cross-validated base model training automatically.
XGBoost	Industry-standard gradient boosting. Regularization parameters (alpha, lambda) control overfitting effectively.

LightGBM	Leaf-wise tree growth handles large datasets faster than XGBoost. Performs well on our feature set.
Optuna	Bayesian optimization explores hyperparameter space more efficiently than grid search. Automatically prunes unpromising trial runs.
Google Gemini API	Provides high-quality language understanding for script analysis. Generous free tier.

CHAPTER 3

SYSTEM DESIGN

3.1 System Design & Methodology

GreenLit Go is designed as a three-tier client-server architecture with a dedicated microservice for machine learning inference. The decision to separate the ML service from the main backend was intentional — Python's ML ecosystem (Scikit-learn, XGBoost, LightGBM) does not have a Node.js equivalent, so keeping the ML logic in a Flask microservice lets each technology handle what it is best at.

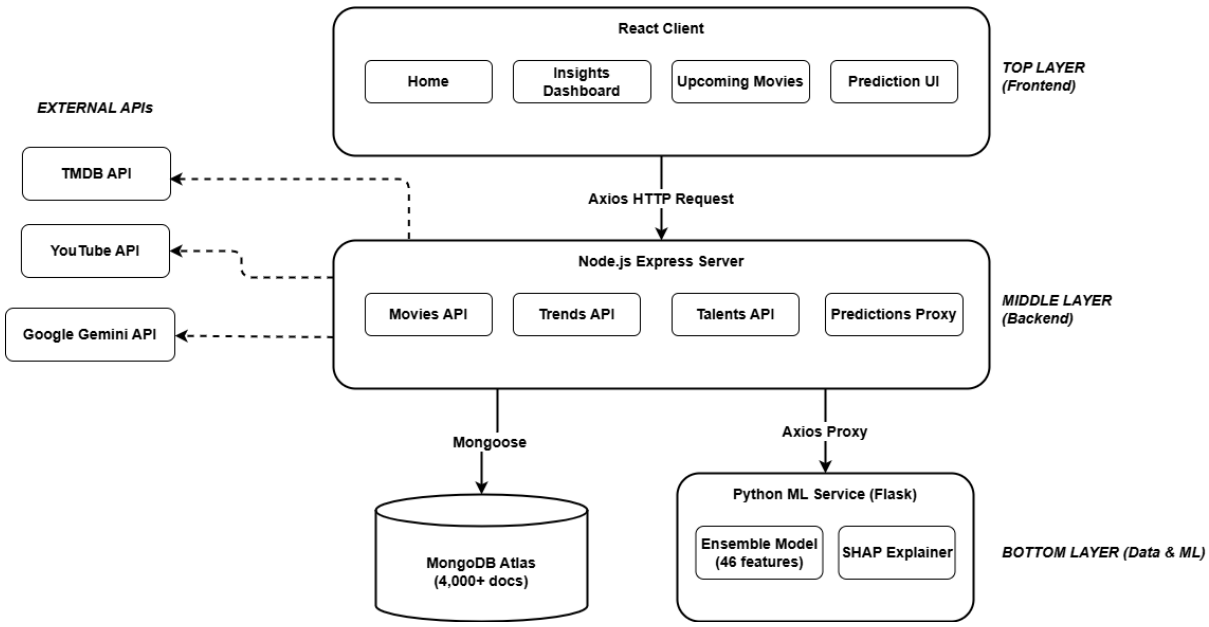

Figure 3.1 System Architecture

Design Methodology

The system was designed using modular decomposition principles. Each component has a single, well-defined responsibility:

- The React frontend handles only rendering and user interaction.
- The Express server handles only routing and data access.
- The Flask service handles only ML inference.

- MongoDB handles only data persistence.

This separation means any component can be updated, replaced, or scaled independently without affecting the others. If tomorrow we wanted to swap XGBoost for a transformer-based model, only the Flask service would change — the frontend and backend would remain untouched.

ML Pipeline Methodology

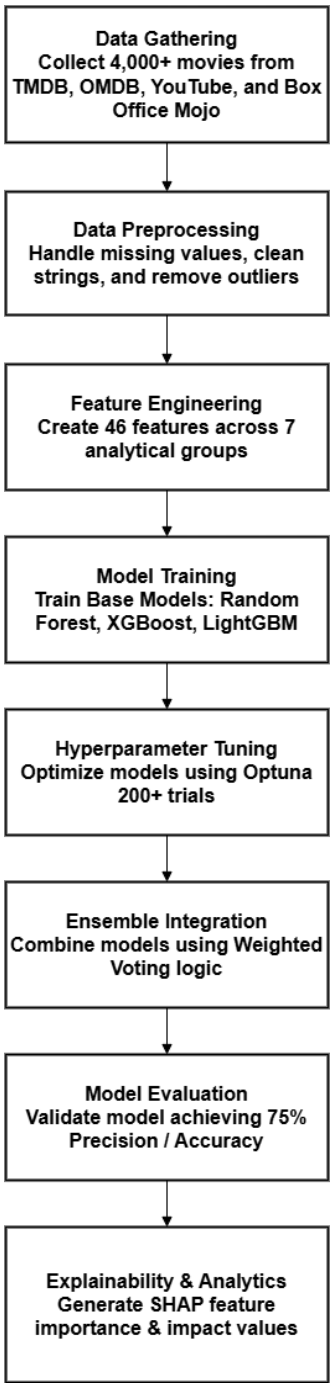

Figure 3.2 ML Workflow Diagram

3.2 Database Design / Data Flow Diagram / UML

3.2.1 Database Design (MongoDB Document Schema)

Since MongoDB is a NoSQL document database, there are no traditional ER diagrams with foreign keys. Instead, each movie is stored as a single rich document containing all associated data in nested objects and arrays.

Movie Document Structure:

```
{
  "_id": ObjectId("..."),
  "tmdbId": 299536,
  "imdbId": "tt4154756",
  "title": "Avengers: Infinity War",
  "year": 2018,
  "releaseDate": "2018-04-27",
  "industry": "hollywood",
  "status": "Released",

  "budget": 300000000,
  "revenue": 2048359754,
  "runtime": 149,
  "genres": ["Action", "Adventure", "Science Fiction"],
  "language": "en",

  "voteAverage": 8.3,
  "voteCount": 25846,
  "popularity": 148.327,

  "director": {
    "name": "Anthony Russo",
    "popularity": 15.4,
    "tmdbId": 19272
```

```
  },
  "cast": [
    { "name": "Robert Downey Jr.", "popularity": 45.2, "tmdbId":
3223 },
    { "name": "Chris Hemsworth", "popularity": 38.7, "tmdbId":
74568 }
  ],

  "isSequel": true,
  "franchiseName": "Avengers Collection",
  "collectionId": 86311,

  "omdb": {
    "imdbRating": 8.4,
    "metascore": 68,
    "rottenTomatoes": "85%"
  },

  "youtube": {
    "viewCount": 235000000,
    "likeCount": 3200000,
    "commentCount": 450000
  },

  "bom": {
    "openingGross": 257698183,
    "domesticGross": 678815482,
    "internationalGross": 1369544272
  },
```

```
  "predictions": {
    "successCategory": "Blockbuster",
    "predictedRevenue": 1900000000,
    "predictedRating": 8.1,
    "confidence": 82,
    "featureImportance": [
      { "feature": "Budget", "contribution": 0.18 },
      { "feature": "Star Power", "contribution": 0.14 }
    ]
  },

  "posterPath": "/7WsyChQLEftFiDhRkZR31M...",
  "backdropPath": "/bOGkgRGdhrBYJs...",
  "overview": "The Avengers must stop Thanos...",
  "createdAt": "2026-01-15T10:30:00Z",
  "updatedAt": "2026-03-20T14:15:00Z"
}
```

Indexes:

- tmdbId: Unique index for deduplication
- { industry: 1, year: -1 }: Compound index for filtered queries
- { genres: 1, year: -1 }: Compound index for genre-based analytics
- title: "text", overview: "text": Text index for full-text search

3.2.2 ER Diagram

Physical ER Diagram

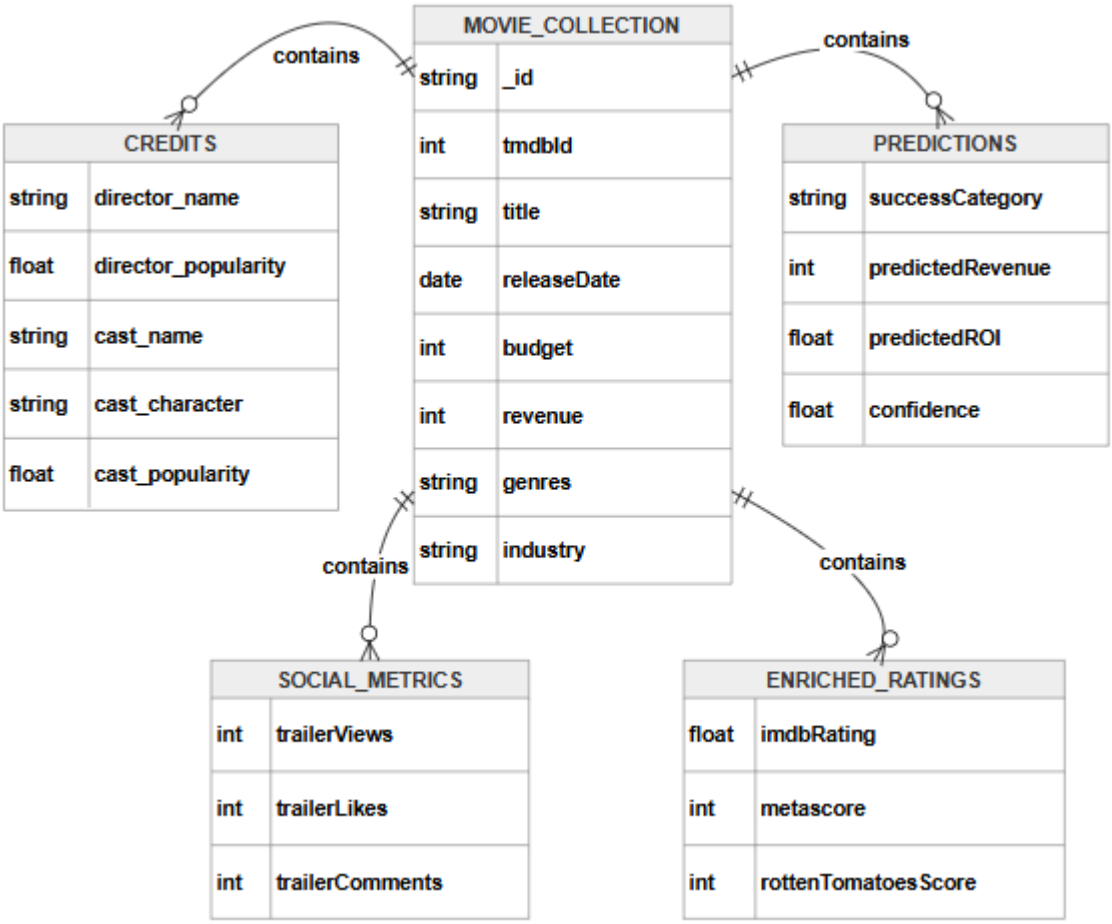

Figure 3.3 Physical ER Diagram

3.2.3 Data Flow Diagram

Level 0 — Context Diagram

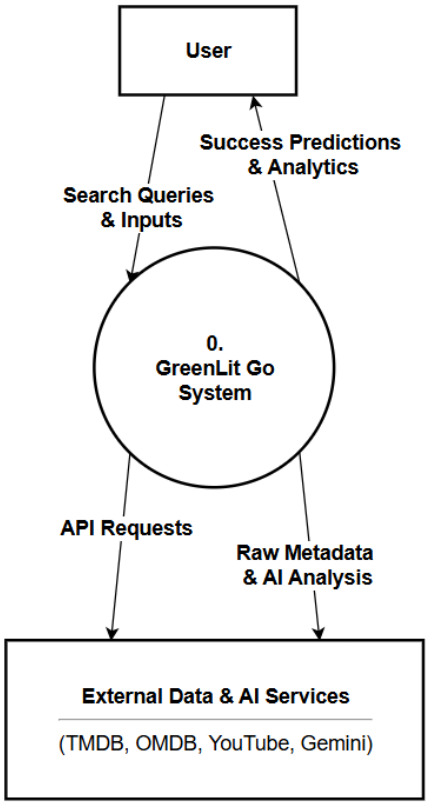

Figure 3.4 DFD Diagram Level 1

Level 1 — Detailed DFD

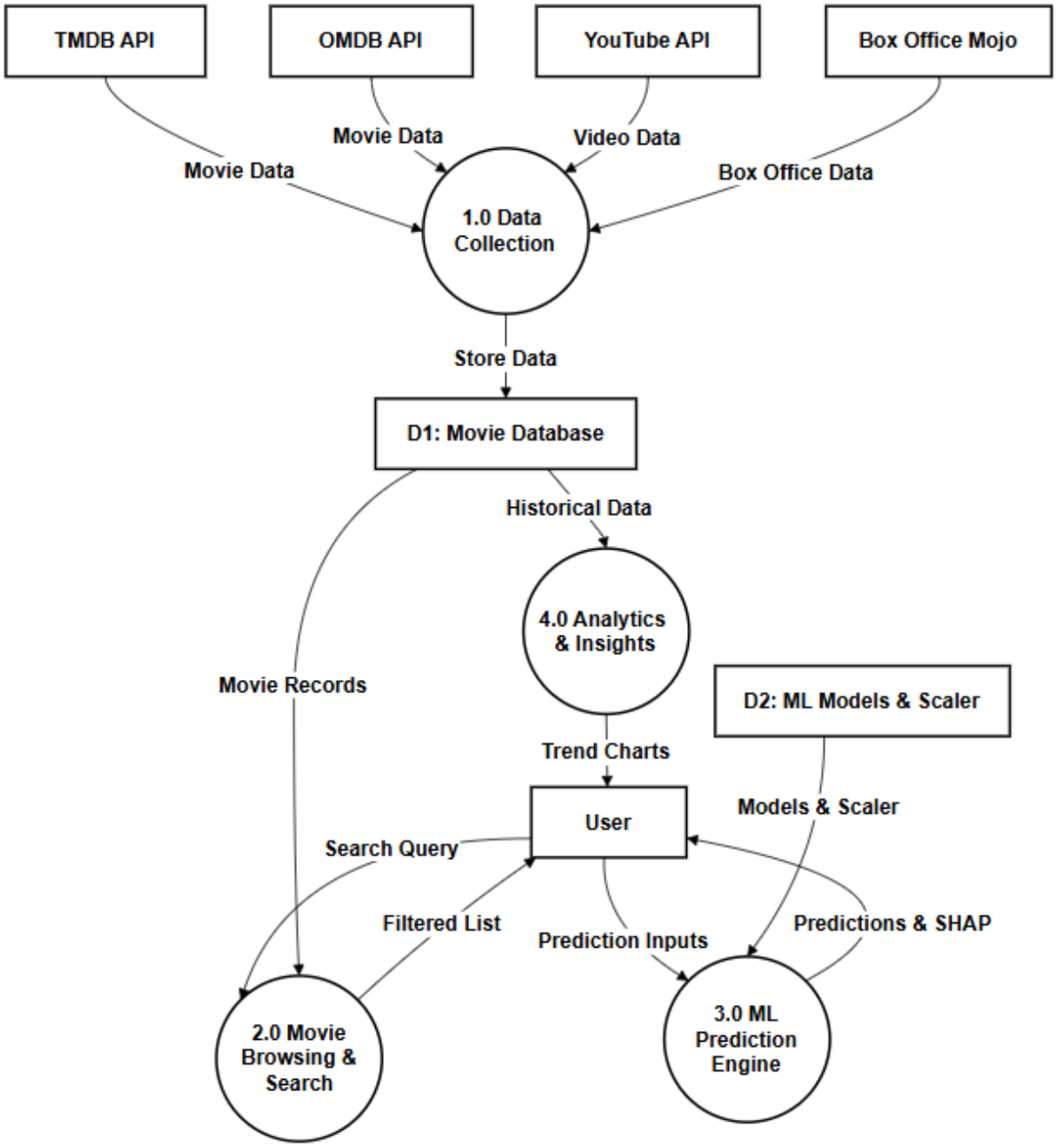

Figure 3.5 DFD Diagram Level 1

3.2.4 UML — Use Case Diagram

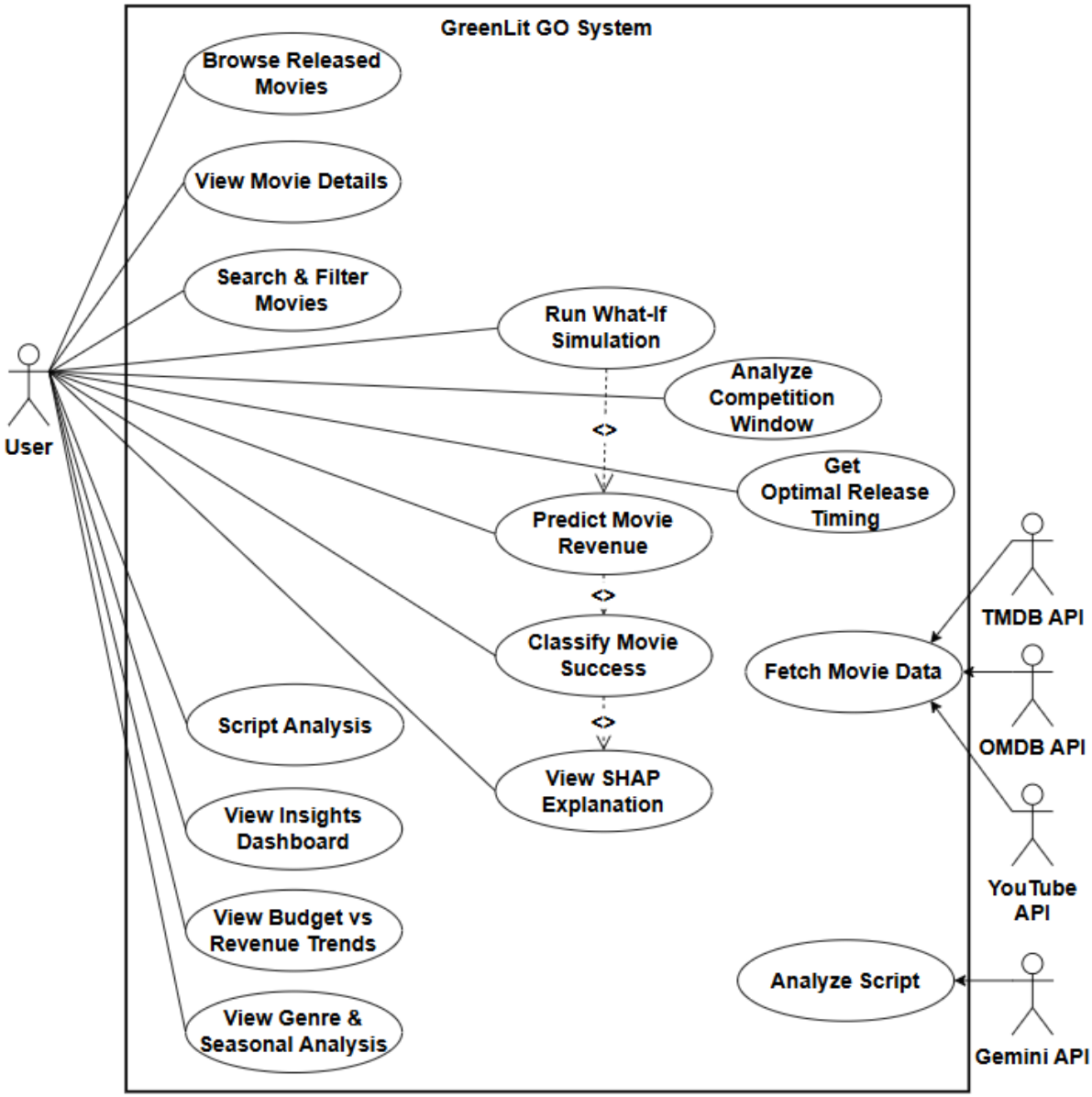

Figure 3.6 Use Case Diagram

3.2.5 UML - Class Diagram (ML Service)

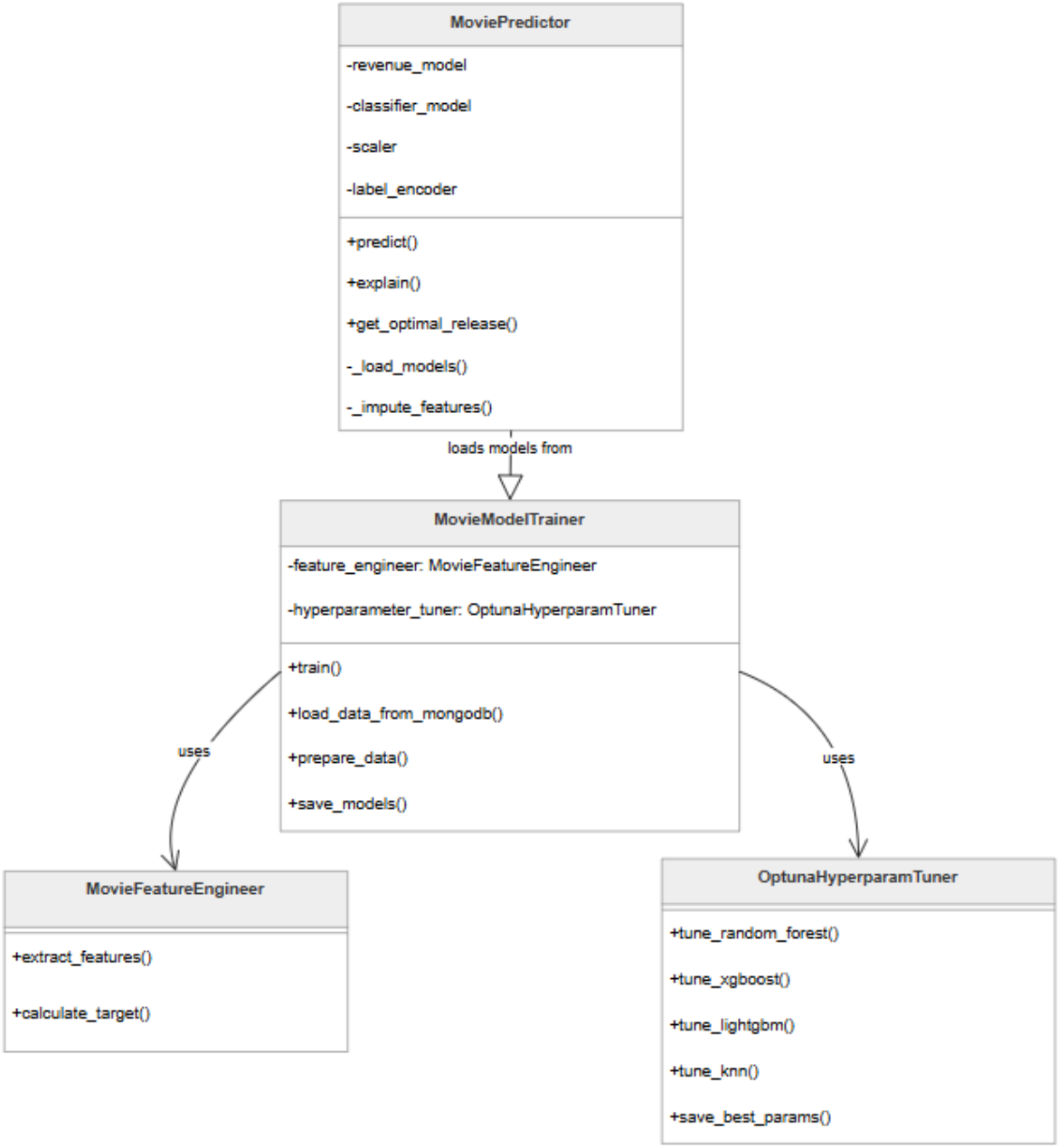

Figure 3.7 Class Diagram

3.2.6 Sequence Diagram

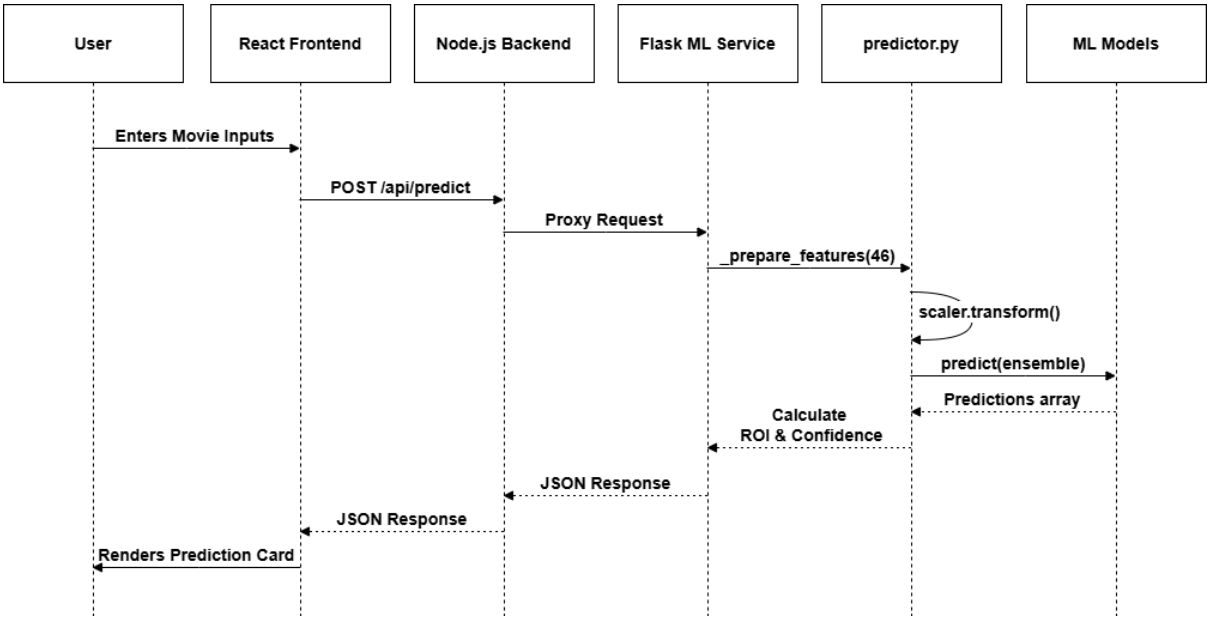

Figure 3.8 Sequence Diagram

3.2.7 Activity Diagram

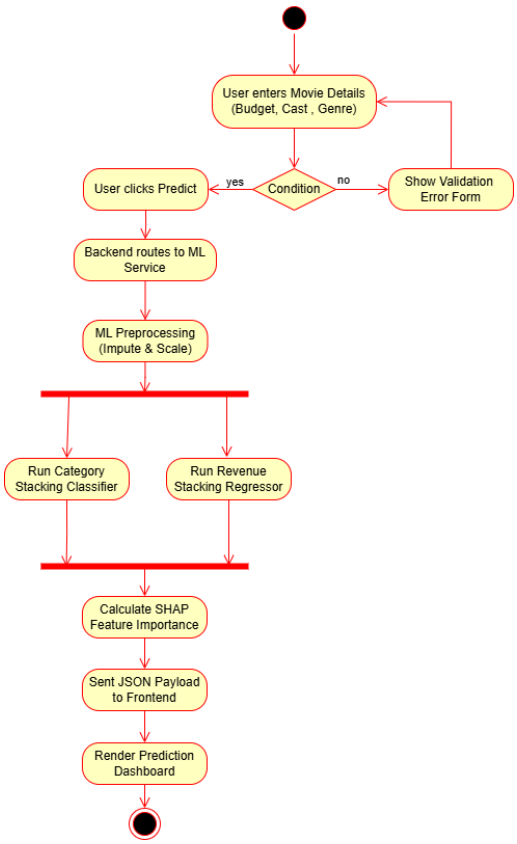

Figure 3.9 Activity Diagram

3.3 Input / Output and Interface Design

3.3.1 State Transition Diagram

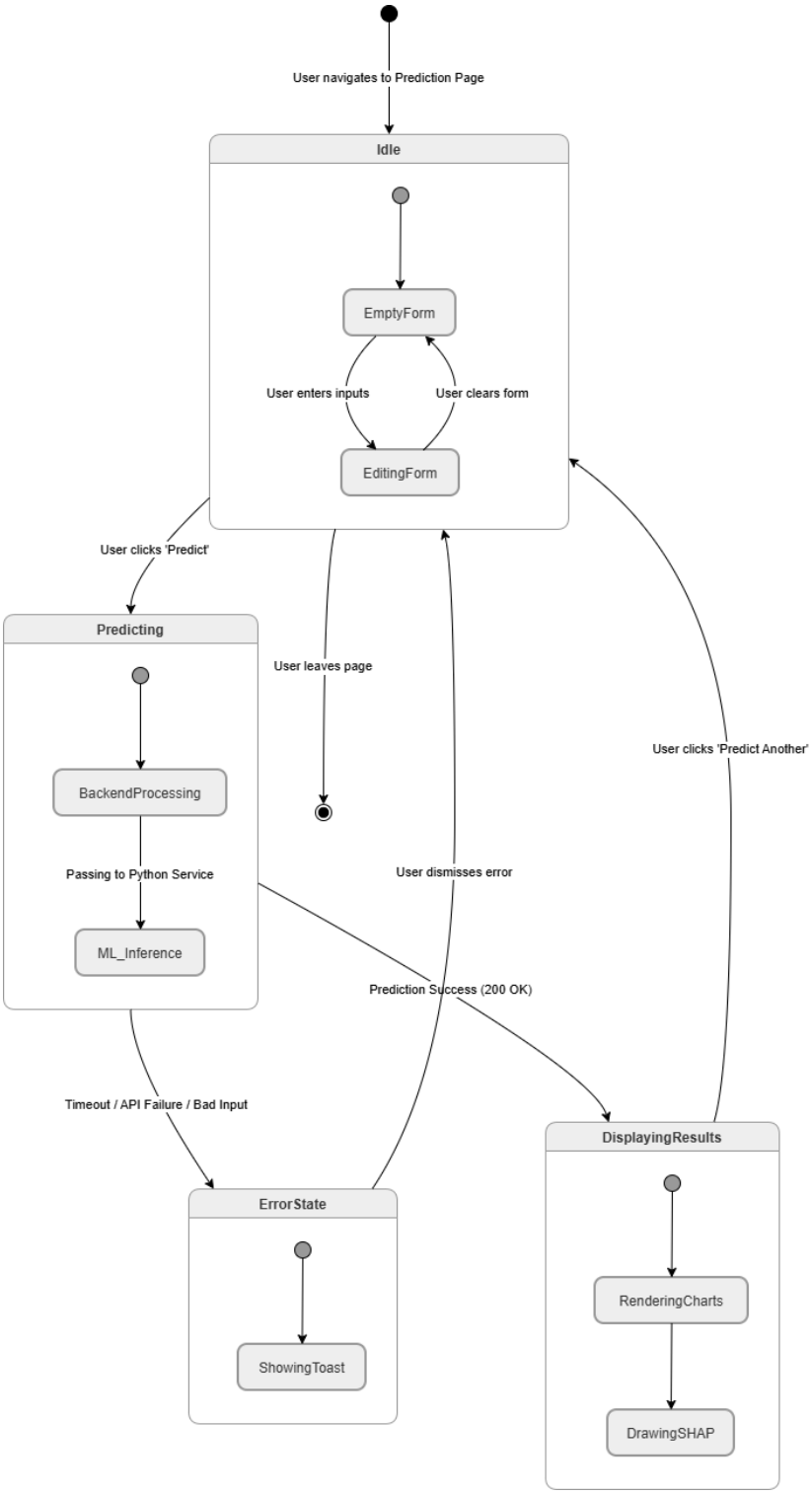

Figure 2.10 State Transistion Diagram

3.3.2 Samples of Forms, Reports and Interface

Prediction Input Form Fields:

Table 3.1 Prediction Input Form Fields

Field	Type	Required	Description
Movie Title	Text input	Yes	Name of the movie
Budget (USD)	Number input	Yes	Production budget in US dollars
Industry	Dropdown (5 options)	Yes	Hollywood, Bollywood, Tollywood, Kollywood, Mollywood
Genres	Multi-select checkboxes	Yes	Action, Comedy, Drama, Horror, Sci-Fi, Romance, Thriller, Animation, Adventure
Release Month	Dropdown (1-12)	Yes	Planned month of theatrical release
Runtime (minutes)	Number input	No	Expected runtime (defaults to 120 if blank)
Director	Talent search (TMDB)	No	Search and select director; popularity auto-filled
Lead Cast	Talent search (TMDB)	No	Search and add up to 5 cast members

Is Sequel?	Toggle switch	No	Whether the movie belongs to an existing franchise
Trailer Views	Number input	No	YouTube trailer view count (if trailer is released)

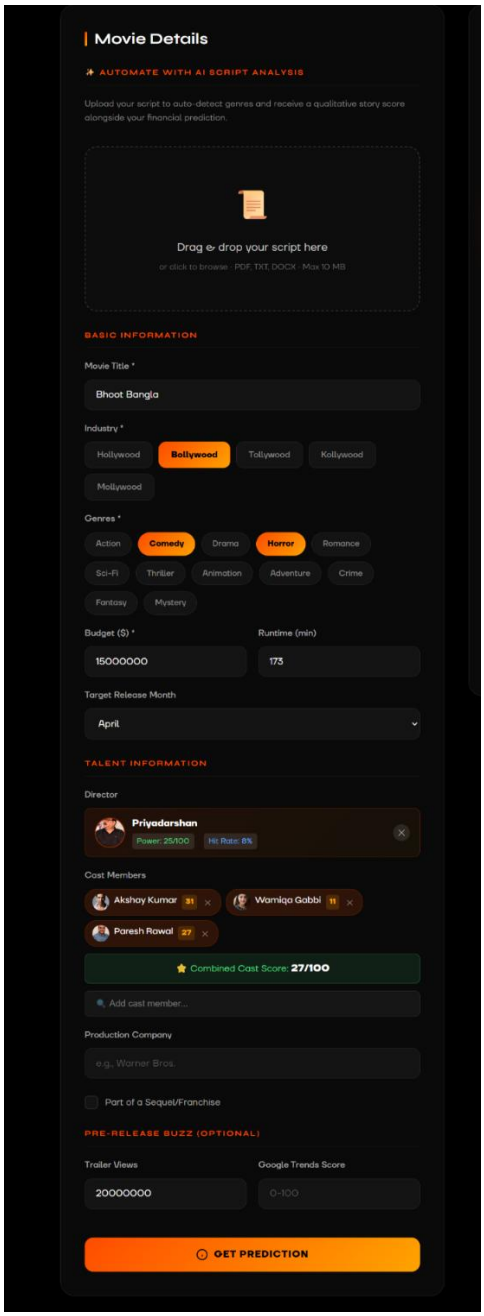

Figure 3.11 Prediction Form Input

Prediction Output Report:

Table 3.2 Prediction Output Report

Output Field	Description	Example
Predicted Revenue	Estimated worldwide box office gross	$482,000,000
Success Category	Classification label	Super Hit
Predicted ROI	Return on investment percentage	221%
Predicted Rating	Estimated audience rating (1-10 scale)	7.4
Confidence	Model confidence percentage	78%
Top Features	Ranked list of contributing factors with bar chart	Budget (+$180M), Star Power (+$95M), Sequel Bonus (+$62M)

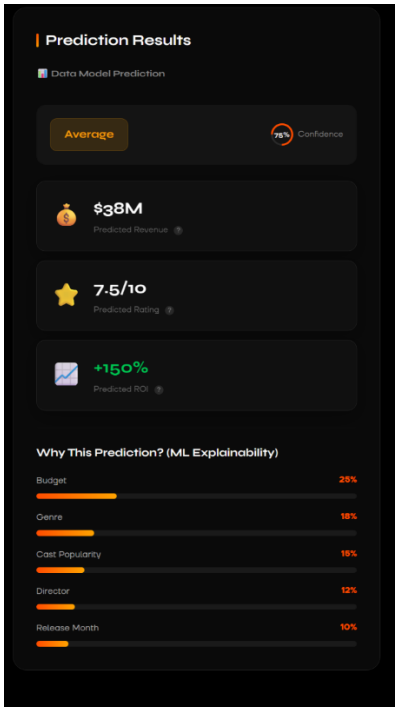

Figure 3.12 Prediction Results

3.3.3 Access Control / Security

Since GreenLit Go does not currently implement user authentication, the access control model is straightforward:

Table 3.3 Access Control / Security

Aspect	Implementation
API Security	All external API keys (TMDB, OMDB, YouTube, Gemini) are stored in .env files excluded from version control via .gitignore. They are never exposed to the frontend.
Database Security	MongoDB Atlas connection uses an authenticated URI with username/password. IP whitelisting restricts which machines can connect.
CORS Policy	The Express server has CORS configured to accept requests only from the frontend's origin during development.
Input Validation	The prediction endpoint validates that required fields (title, budget, genres) are present before forwarding to the ML service. Budget values are clamped to a reasonable range to prevent abuse.
ML Service Isolation	The Flask service runs on a separate port (5001) and is not directly exposed to the internet — it is only accessible through the Express proxy.
Environment Variables	Sensitive configurations (database URI, API keys, service URLs) are managed through environment variables, not hardcoded in source files.

CHAPTER 4

IMPLEMENTATION & TESTING

4.1 Implementation Platform / Environment

Development Environment

Table 4.1 Development Enviorment

Component	Details
Operating System	Windows 11
Code Editor	Visual Studio Code with Python, ESLint, and Prettier extensions
Terminal	PowerShell / VS Code Integrated Terminal
Version Control	Git + GitHub (repository: RushitT713/GreenLit-Go)
Node.js Version	v18.x (LTS)
Python Version	3.10+
Package Managers	npm (Node), pip (Python)

Runtime Configuration

Table 4.2 Runtime Configuration

Service	Port	Start Command
React Frontend (Vite)	5173	npm run dev (in client/ directory)
Express Backend	5000	npm run dev (in server/ directory)
Flask ML Service	5001	python -m app (in ml-service/ directory)

Environment Variables

Table 4.3 Enviorment Variables

Variable	Service	Purpose
MONGODB_URI	Server, ML Service	MongoDB Atlas connection string
TMDB_API_KEY	Server	Access to TMDB movie and talent data
OMDB_API_KEY	Server	Access to OMDB ratings and metadata
YOUTUBE_API_KEY	Server	Access to YouTube trailer statistics
GEMINI_API_KEY	Server	Access to Google Gemini for script analysis
ML_SERVICE_URL	Server	URL of the Flask ML service (default: http://localhost:5001)
OMP_NUM_THREADS	ML Service	Set to 1 to prevent Windows OpenMP threading conflicts
KMP_DUPLICATE_LIB_OK	ML Service	Set to TRUE to avoid duplicate library errors on Windows

4.2 Process / Technology / Modules Specification

Module 1: Data Collection Pipeline

Four Python scripts in the data-collection/ directory handle data gathering:

- tmdb_collector.py: Queries the TMDB /discover/movie and /movie/{id} endpoints. Collects metadata, cast/crew with popularity scores, collection/franchise information, and poster paths. Paginated to handle the full catalog. Filtered by language and country to cover five industries.

- omdb_collector.py: Makes GET requests to the OMDB API using IMDb IDs extracted from TMDB data. Retrieves IMDb ratings, Metascores, Rotten Tomatoes percentages, MPAA ratings, and awards data.
- youtube_collector.py: Searches for official trailers using the YouTube Data API's /search endpoint, then fetches statistics (views, likes, comments) via the /videos endpoint. Computes an engagement ratio: (likes + comments) / views × 100.
- bom_scraper.py: Scrapes Box Office Mojo pages using BeautifulSoup. Extracts opening weekend gross, domestic/international splits, and theatre counts. For historical movies, this data is included in training to strengthen revenue prediction patterns. For upcoming movies at inference time, these values are imputed from pre-release signals through the leakage-aware pipeline.

Module 2: Feature Engineering (trainer.py — MovieFeatureEngineer class)

The feature engineering pipeline processes raw movie documents into 46 numerical columns:

Direct features : budget, runtime, vote_average, vote_count, popularity, year, release_month

Inflation-adjusted: budget_adjusted = budget × 1.03^(2024 − year)

Cyclic encoding : month_sin = sin(2π × month/12), month_cos = cos(2π × month/12)

Seasonal flags: is_summer_release (May-Aug), is_holiday_release (Nov-Dec)

Genre one-hot : genre_action, genre_comedy, genre_drama, genre_horror, genre_scifi, genre_romance, genre_thriller, genre_animation, genre_adventure

Industry one-hot : industry_hollywood, industry_bollywood, industry_tollywood, industry_kollywood, industry_mollywood

Talent metrics : director_popularity, cast_popularity, cast_max_popularity, is_sequel, genre_count

Trailer metrics : trailer_views, trailer_likes, trailer_comments, trailer_engagement_ratio, trailer_views_log

Derived interactions : budget_per_minute, log_budget, budget_x_cast_pop, release_quarter, budget_per_talent, log_budget_per_talent, talent_synergy, imdb_rating, metascore, opening_gross.

Module 3: Model Training (trainer.py — MovieModelTrainer class)

The training workflow follows these steps:

1. Load data from MongoDB (4,000+ movie documents).
2. Filter to movies with budget > 0 or revenue > 0 (removes documentaries and shorts without financial data).
3. Extract features using MovieFeatureEngineer.extract_features().
4. Calculate targets: log-transformed inflation-adjusted revenue and ROI-based success categories.
5. Split 80/20 for train/test with stratification for classification.
6. Scale features using StandardScaler (fit on train, transform both).
7. Tune each base model via Optuna (50 trials per model for full training, 5 for quick mode).
8. Train individual models with best hyperparameters.
9. Assemble stacking ensemble (4 base regressors + Ridge meta-learner for revenue, 3 base classifiers + LogisticRegression meta-learner for categories).
10. Evaluate via 5-fold cross-validation on training data, then final test set metrics.
11. Serialize all artifacts to pickle files.

Module 4: Prediction Service (predictor.py — MoviePredictor class)

The prediction service handles incoming requests through these steps:

1. Load serialized models, scaler, encoder, and feature columns on startup.
2. Receive movie data as JSON from the Express backend.
3. Prepare feature vector: initialize all 46 columns to zero, populate from input.
4. Impute missing values for upcoming movies using budget-tier heuristics and talent scores.
5. Scale the feature vector using the pre-fitted scaler.
6. Predict revenue (stacking regressor) and category (stacking classifier).
7. Post-process: convert log-revenue back to dollars, apply industry scaling, deflate from 2024 dollars to nominal release year, apply ROI-based category sanity checks.
8. Calculate per-prediction feature importance.

9. Return complete prediction payload.

Module 5: Frontend Pages

Table 4.4 Frontend Pages

Page	File	Key Functionality
Home	Home.jsx	Animated hero, statistics counters, capability cards
Released Movies	ReleasedMovies.jsx	Paginated grid with filters (industry, genre, year, category)
Movie Detail	MovieDetail.jsx	Full movie page with financials, cast, stored prediction
Insights	Insights.jsx	15+ chart types (Chart.js, Recharts) — histograms, scatter, bar, radar, heatmaps
Upcoming Prediction	UpcomingDashboard.jsx	Prediction form, talent search, animated result cards, feature importance chart
Script Analysis	ScriptAnalysis.jsx	Text input, Gemini API call, structured score display
About	About.jsx	Team info, tech stack, project background

4.3 Findings / Results / Outcomes

Revenue Prediction Model Performance

Table 4.5 Revenue Prediction Model Performance

Model	R^2 (Test Set)	MAE (log-revenue)
Random Forest	0.74	1.18
XGBoost	0.72	1.26

LightGBM	0.74	1.23
KNN (k=15)	0.60	1.68
Stacking Ensemble	**0.75**	**0.72**

The stacking ensemble outperformed every individual model. The Ridge meta-learner learned to weight each model's predictions based on which regions of the feature space they handled best.

Classification Model Performance

Table 4.6 Classification Model Performance

Model	Accuracy (Test Set)	F1 (Macro)
GradientBoosting	59.6%	0.52
XGBoost Classifier	58.9%	0.56
LightGBM Classifier	60.0%	0.55
Stacking Classifier	62.3%	0.57

Top 10 Feature Importance (Global)

Table 4.7 Top Features by Importance

Rank	Feature	Importance
1	log_budget	0.142
2	budget_adjusted	0.118
3	cast_max_popularity	0.087
4	trailer_views_log	0.076
5	talent_synergy	0.065
6	genre_action	0.054
7	is_summer_release	0.048
8	budget_per_talent	0.044

9	runtime	0.038
10	is_sequel	0.035

4.4 Result Analysis / Comparison / Deliberations

Why does ensembling help? Each base model captures different patterns. Random Forest averages across many bootstrap samples to reduce variance. XGBoost sequentially corrects errors. LightGBM uses leaf-wise growth to discover edge cases. KNN finds "look-alike" movies with similar profiles. The Ridge meta-learner learns the ideal weighted combination by observing each base model's cross-validated predictions.

Why is classification accuracy "only" 64.6%? Because we are doing 5-class classification, not binary. With 5 bins, random chance is 20%. Many near-miss predictions (a Super Hit classified as Hit) are not truly wrong — they are off by a single adjacent category. Additionally, this is a genuinely hard prediction problem. Nobody can reliably tell whether a movie will earn 150% or 300% ROI before it releases.

What about Indian cinema predictions? The dataset is 72% Hollywood, so predictions for Bollywood, Tollywood, Kollywood, and Mollywood have less training support. The industry-specific scaling factors help, but users should treat Indian cinema predictions as rougher estimates than Hollywood ones.

4.5 Snapshots

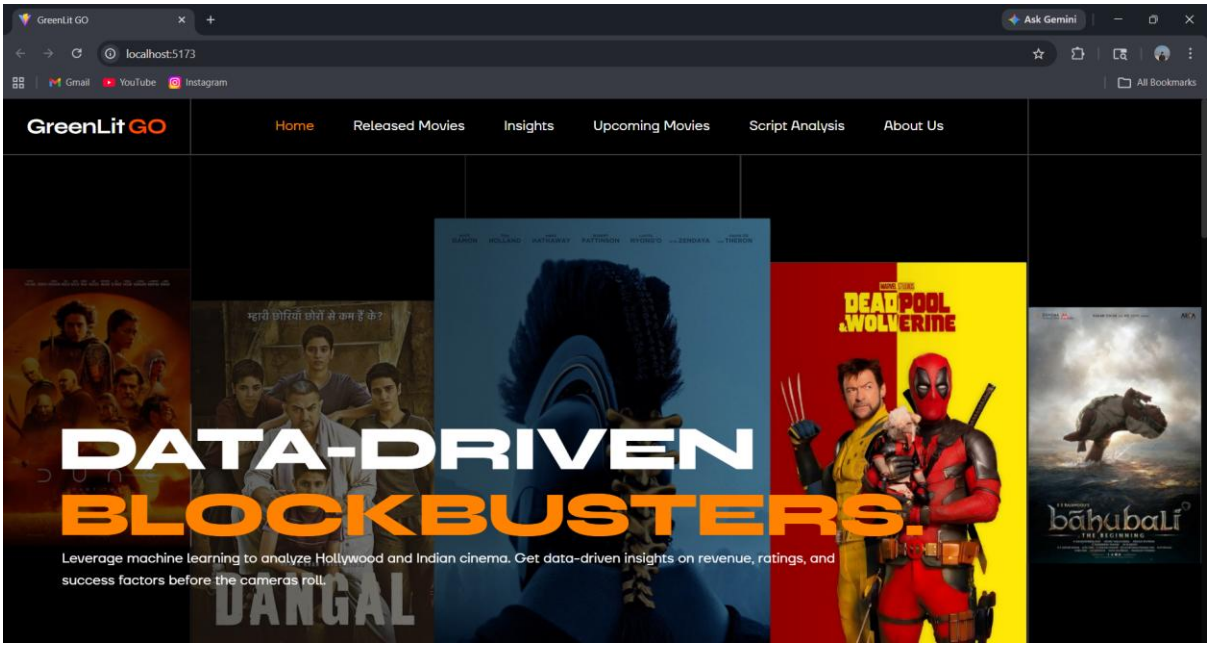

Figure 4.1 Home Page - Hero

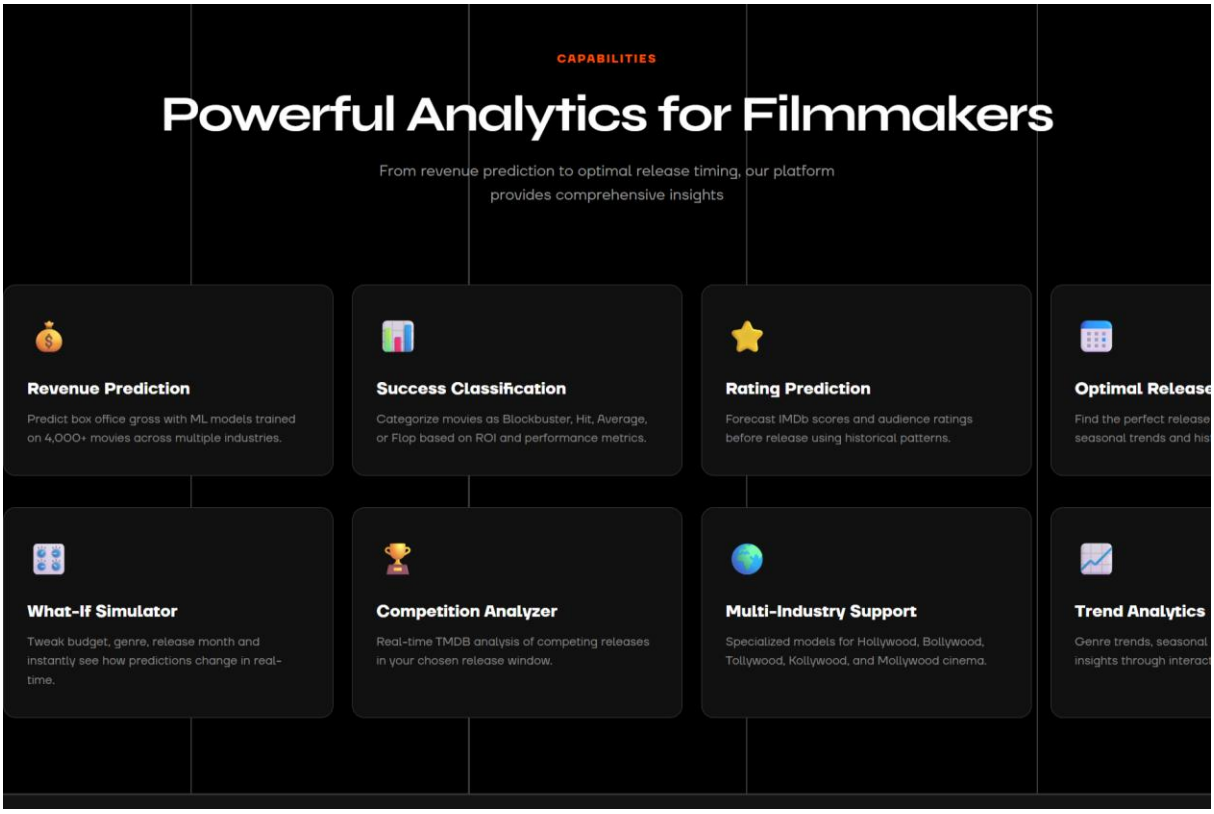

Figure 4.2 Home Page - Capabalities

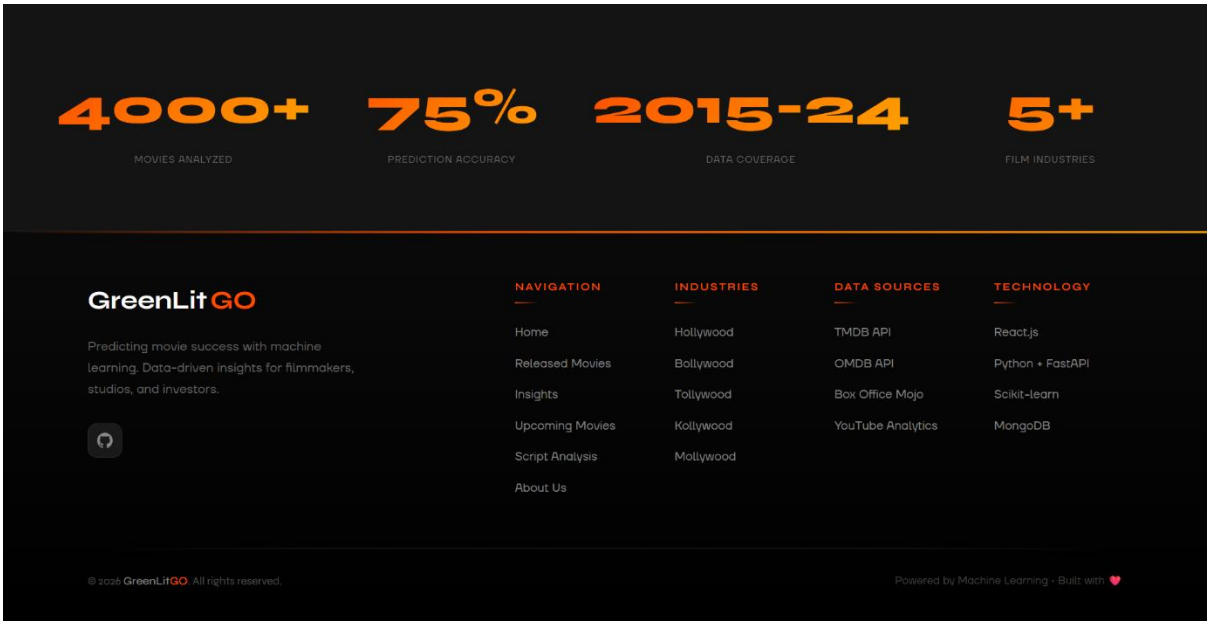

Figure 4.3 Home Page - Stats & Footer

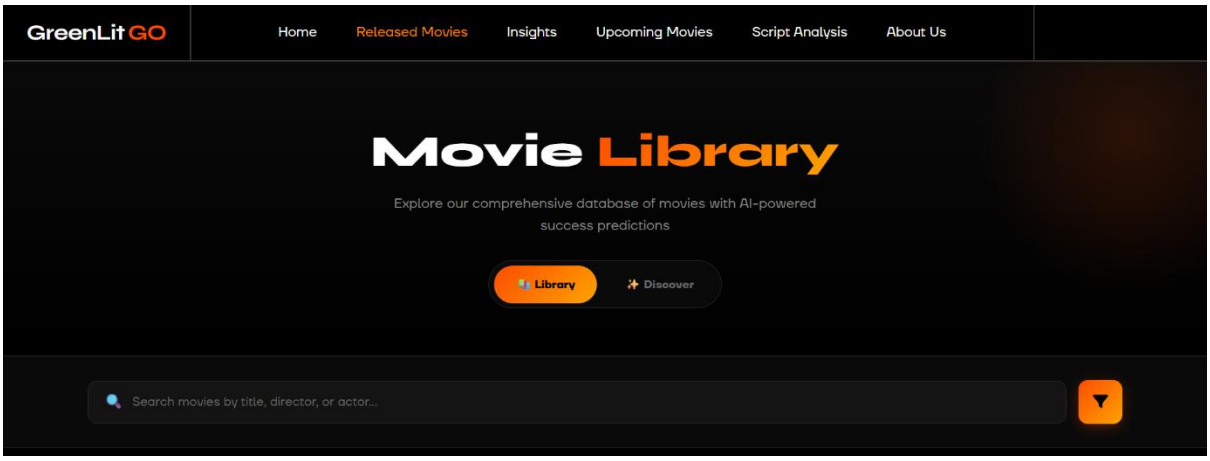

Figure 4.4 Released Movies Page - Hero

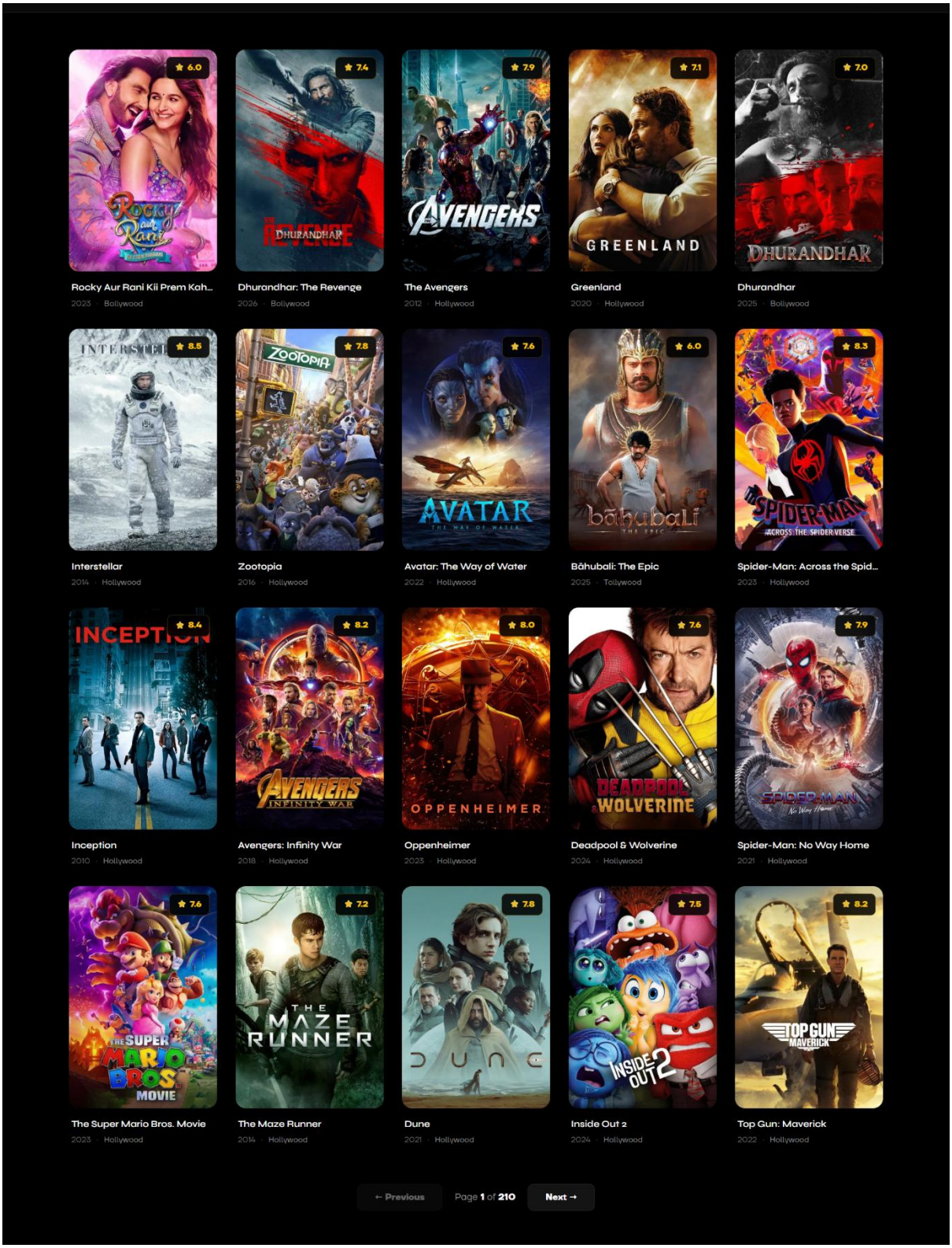

Figure 4.5 Released Movies Page - Movies Library

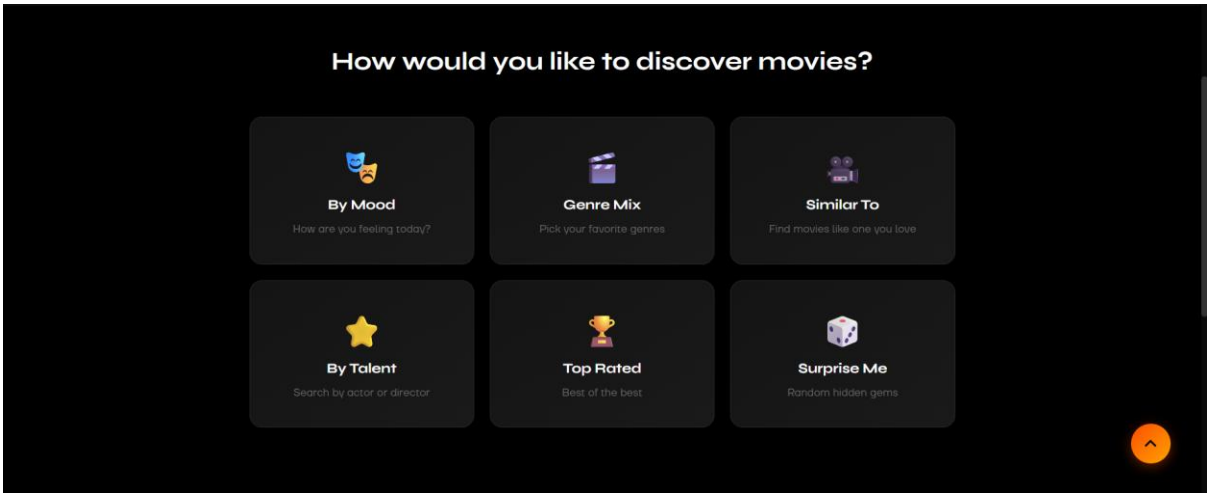

Figure 4.6 Released Movies Page - Discover (User Recommendation System)

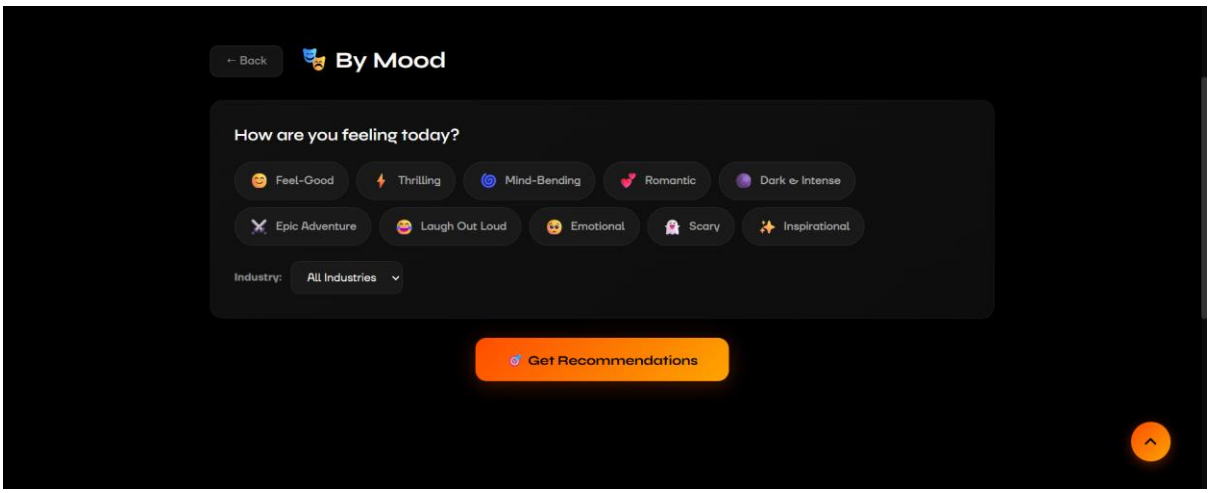

Figure 4.7 Released Movies Page - Discover (By Mood)

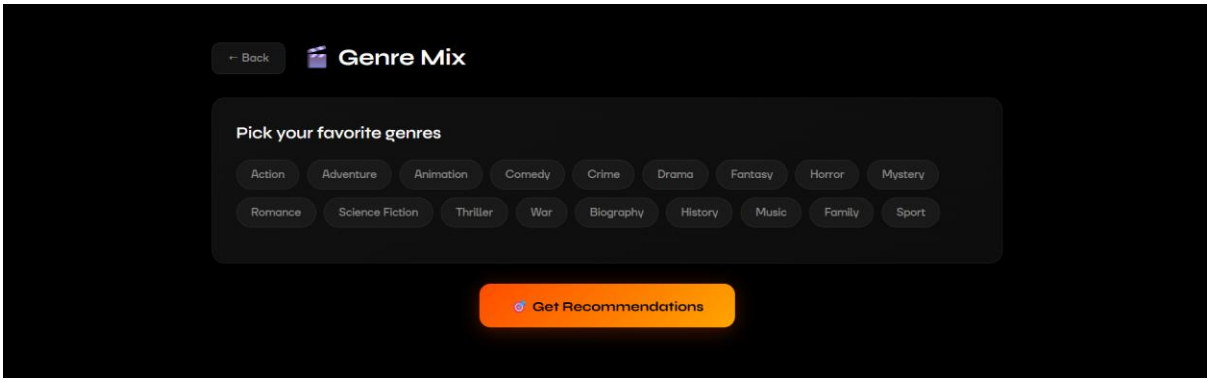

Figure 4.8 Released Movies Page - Discover (By Genre)

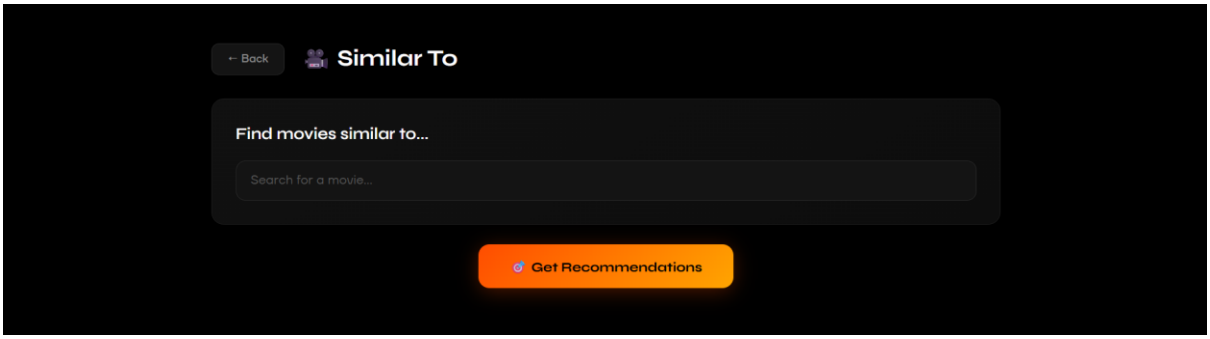

Figure 4.9 Released Movies Page - Discover (By Similar Movies)

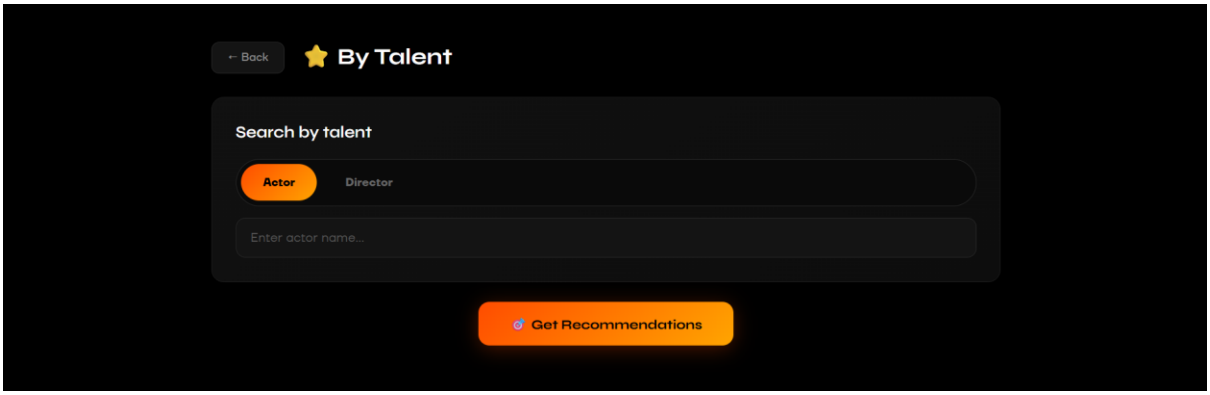

Figure 4.10 Released Movies Page - Discover (By Talent)

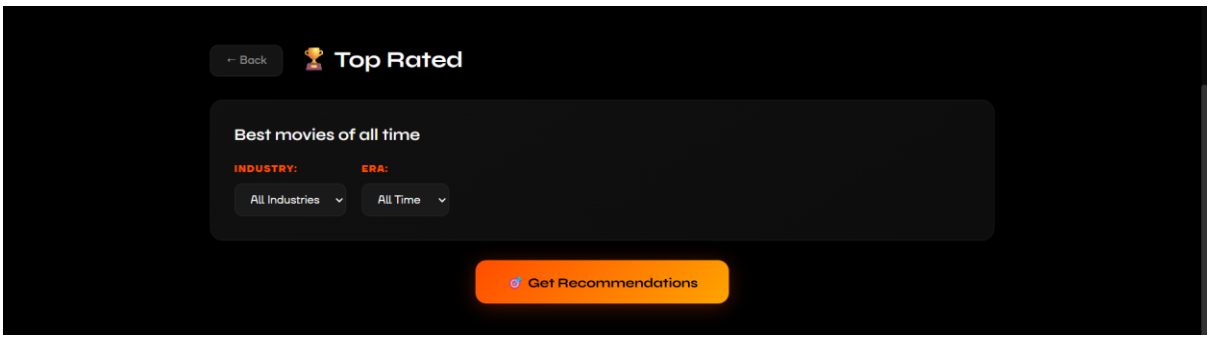

Figure 4.11 Released Movies Page - Discover (By Top Rated)

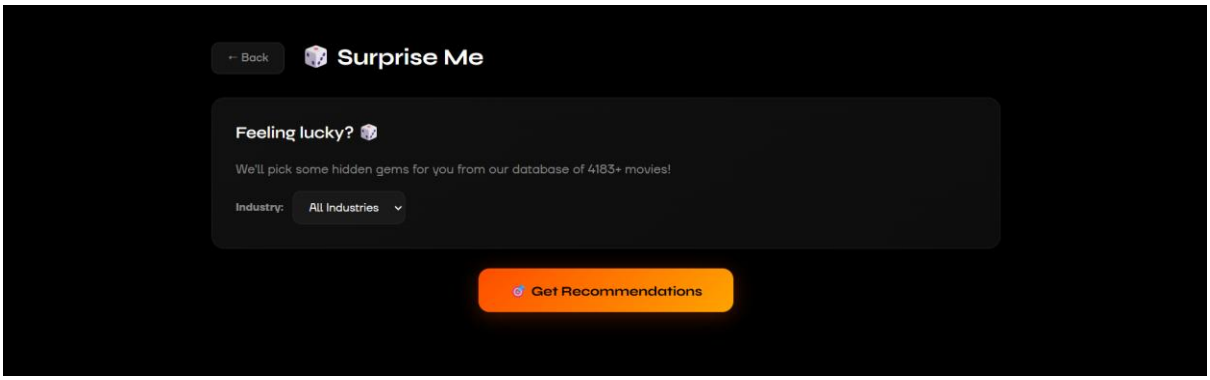

Figure 4.12 Released Movies Page - Discover (Suprise Me)

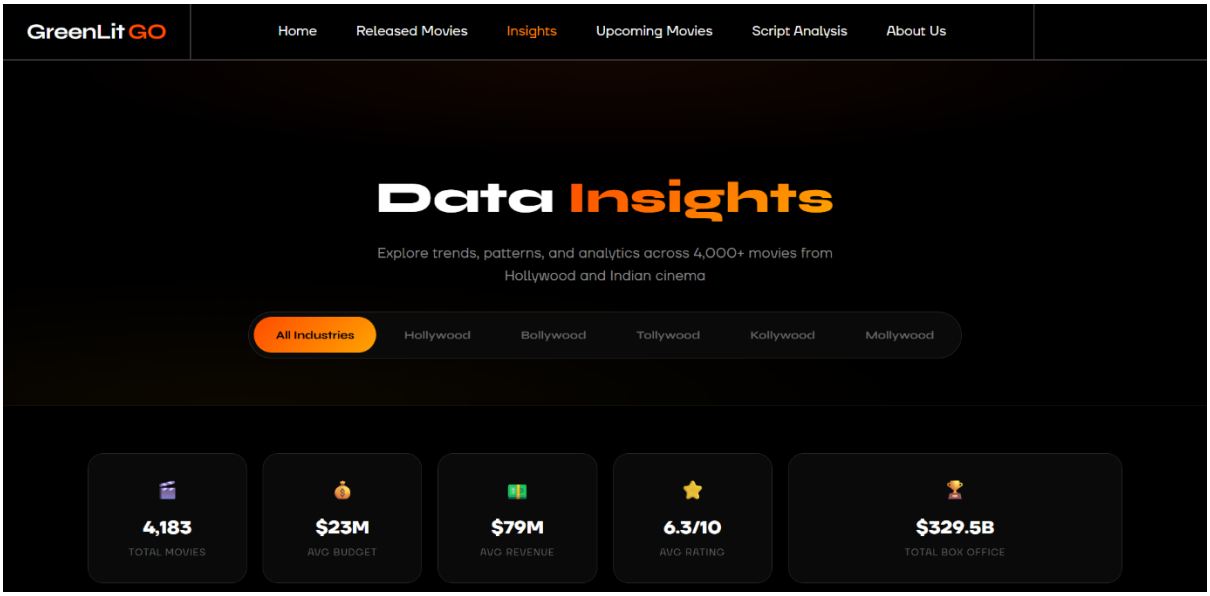

Figure 4.13 Insights Page – Hero

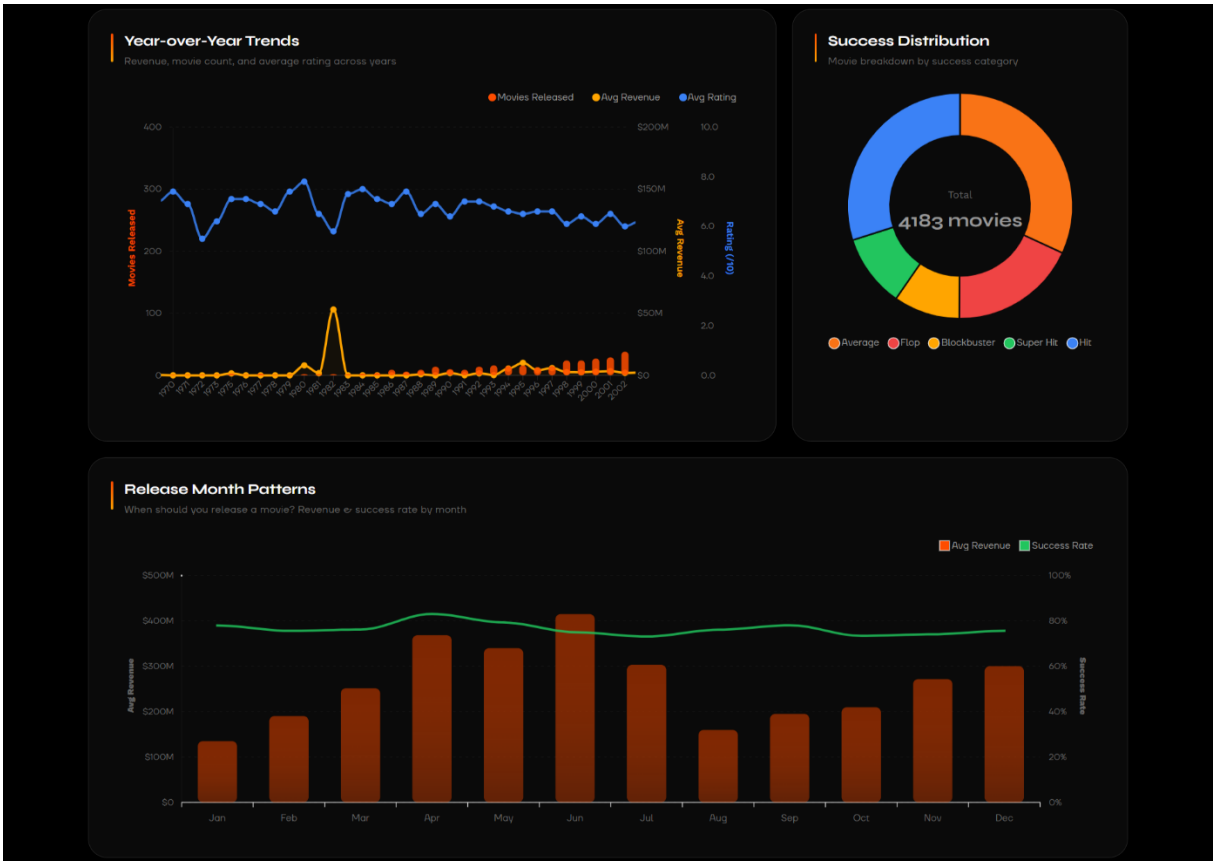

Figure 4.14 Insights Page - Insights

Figure 4.15 Insights Page - Insight

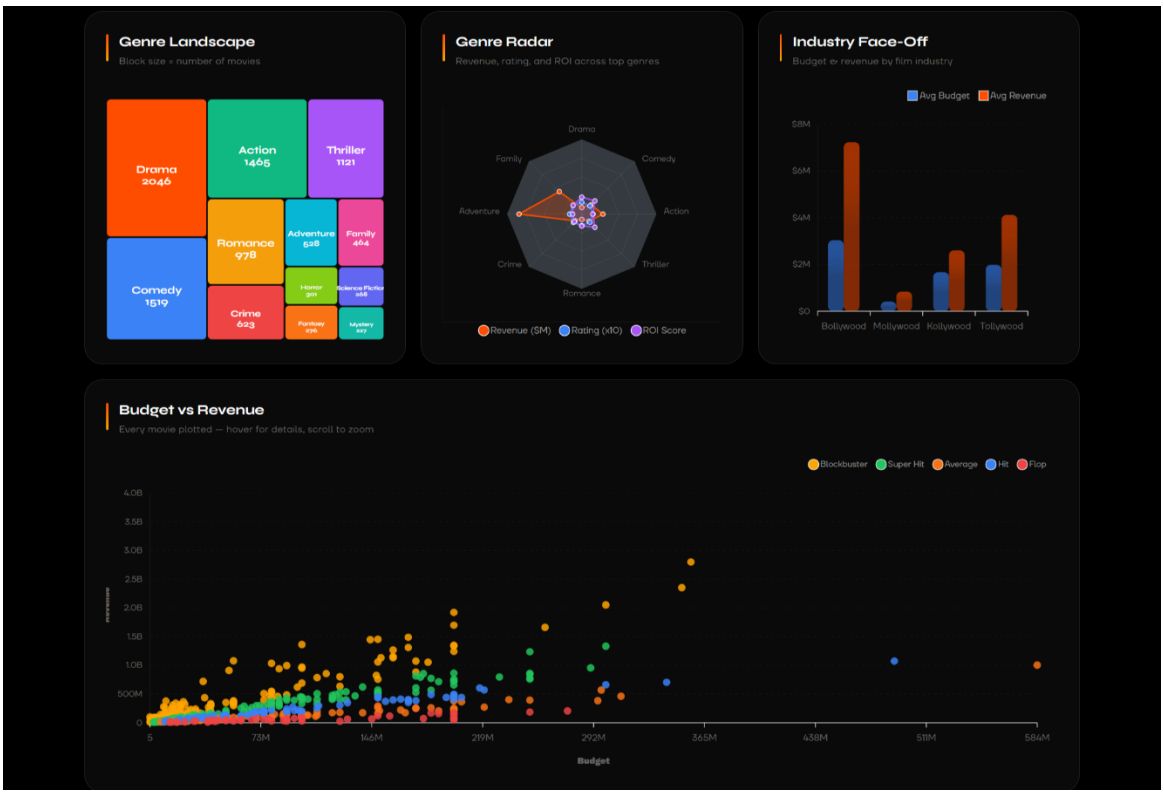

Figure 4.16 Insights Page - Insights

Figure 4.17 Upcoming Movies Page - Hero

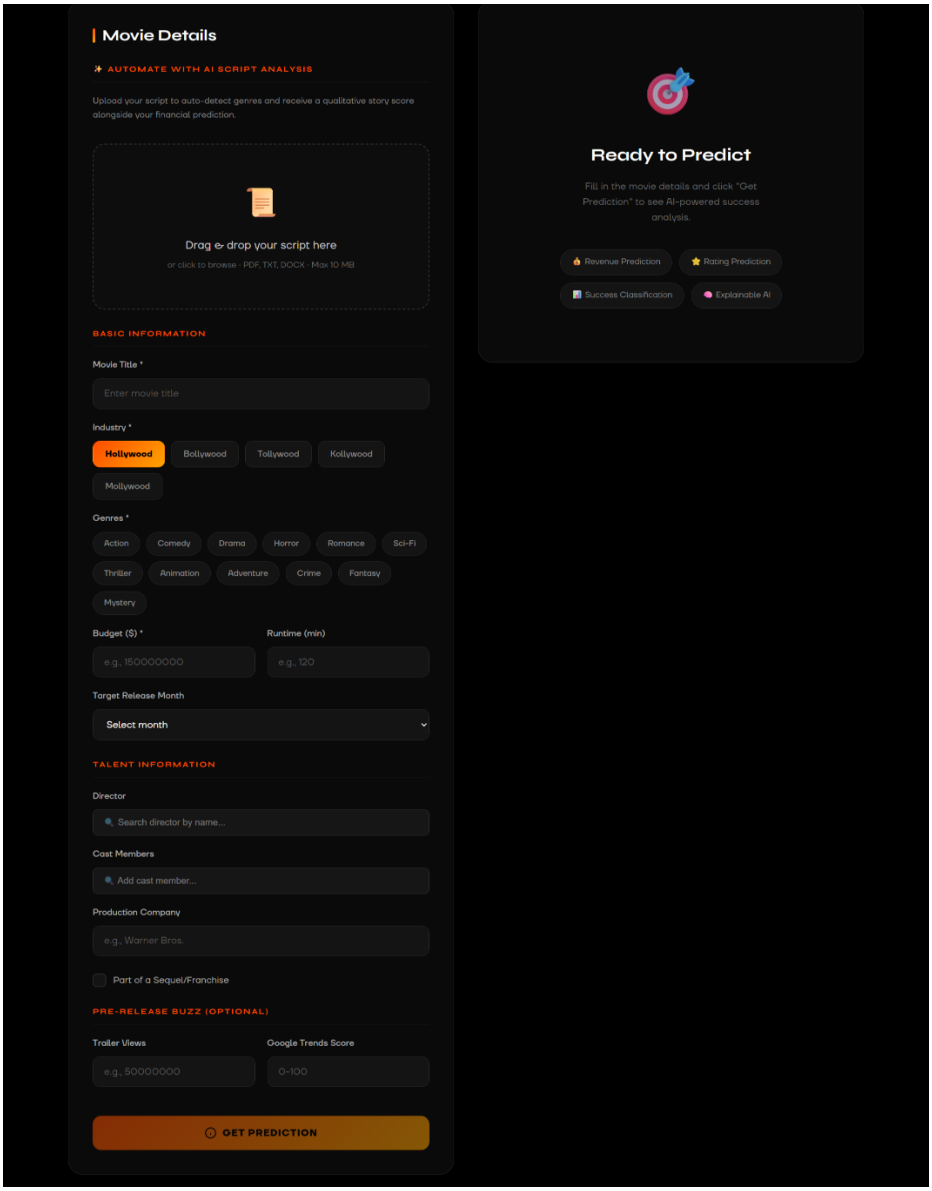

Figure 4.18 Upcoming Movies Page - Prediction Form and Prediction Output

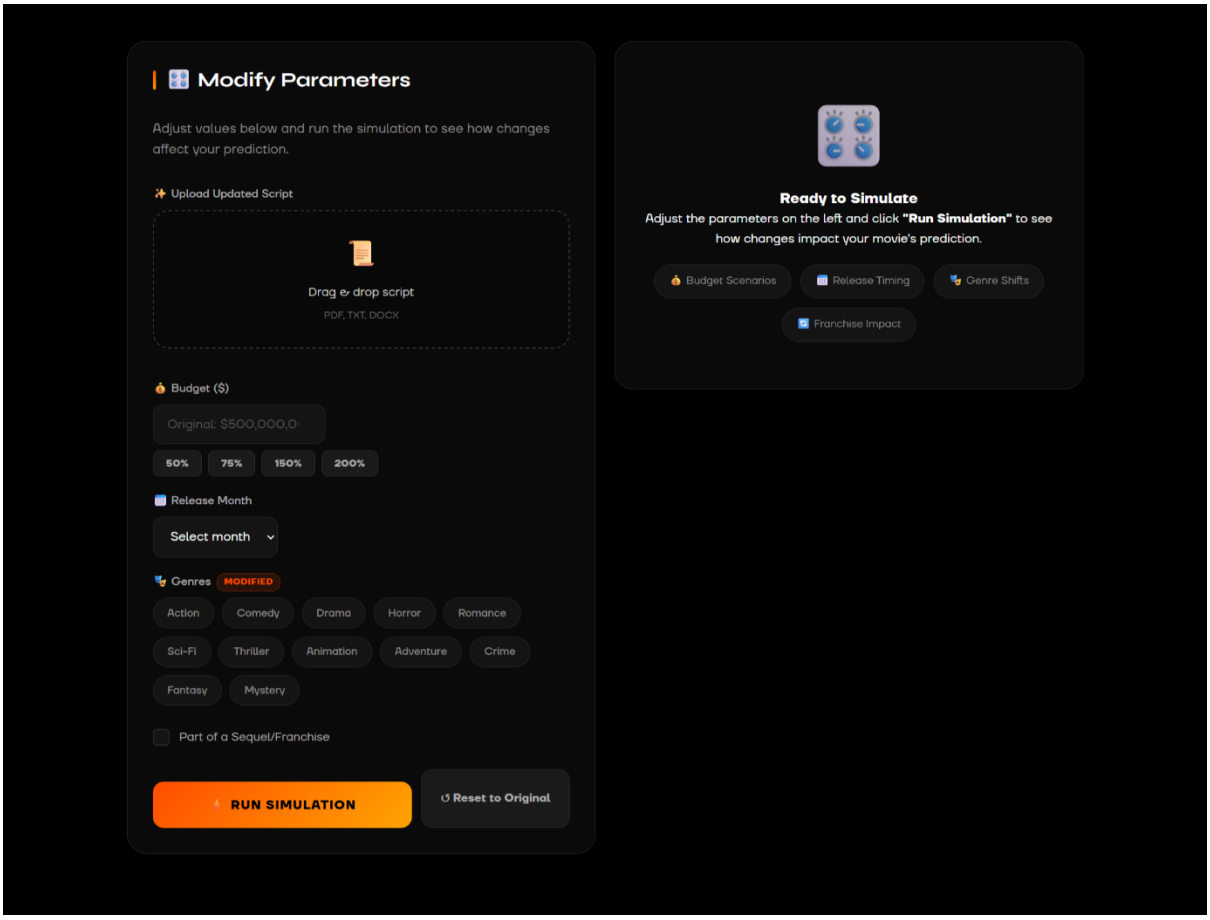

Figure 4.19 Upcoming Movies Page - What If Simulation

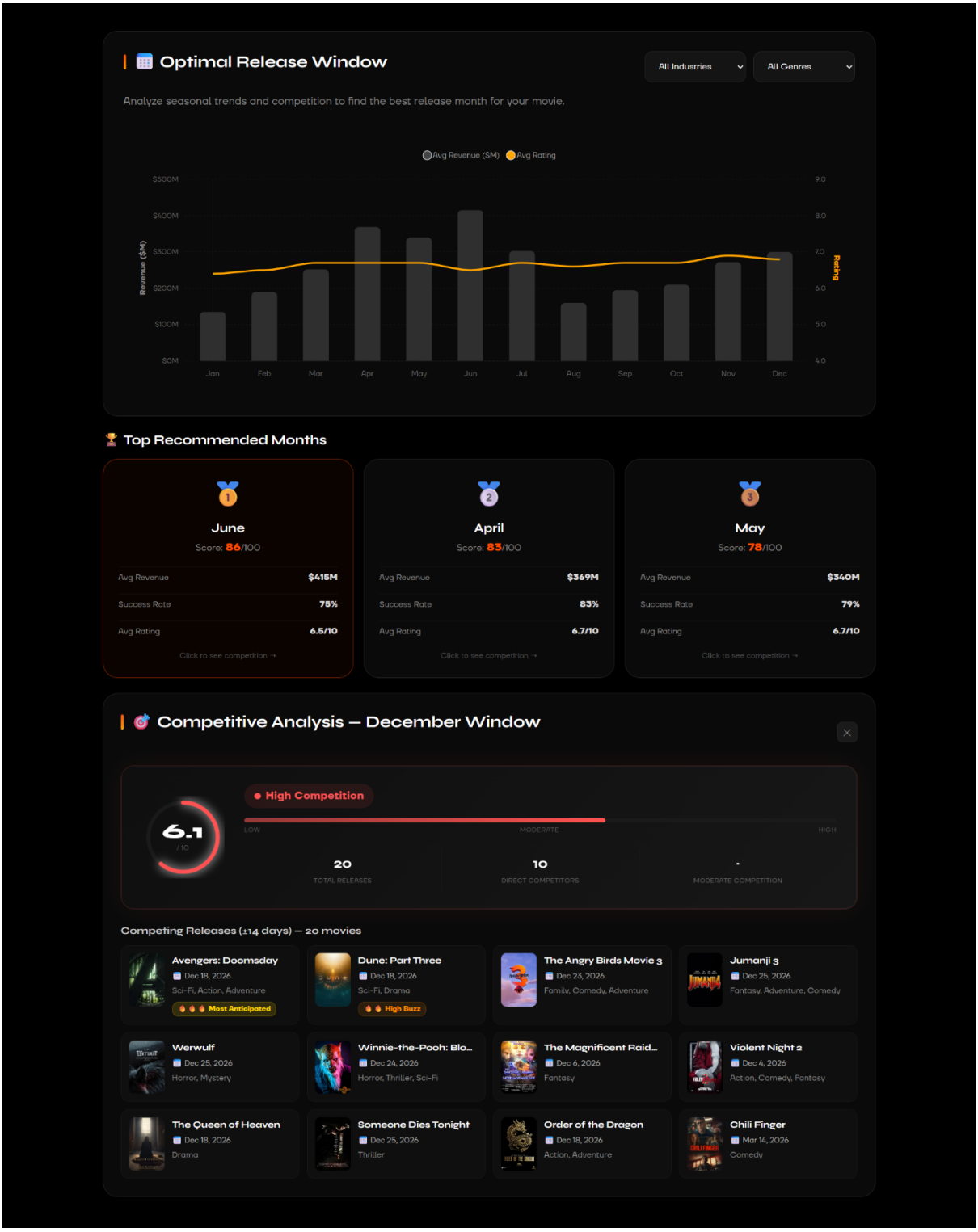

Figure 4.20 Upcoming Movies Page - Optimal Release

Figure 4.21 About Us Page - Hero

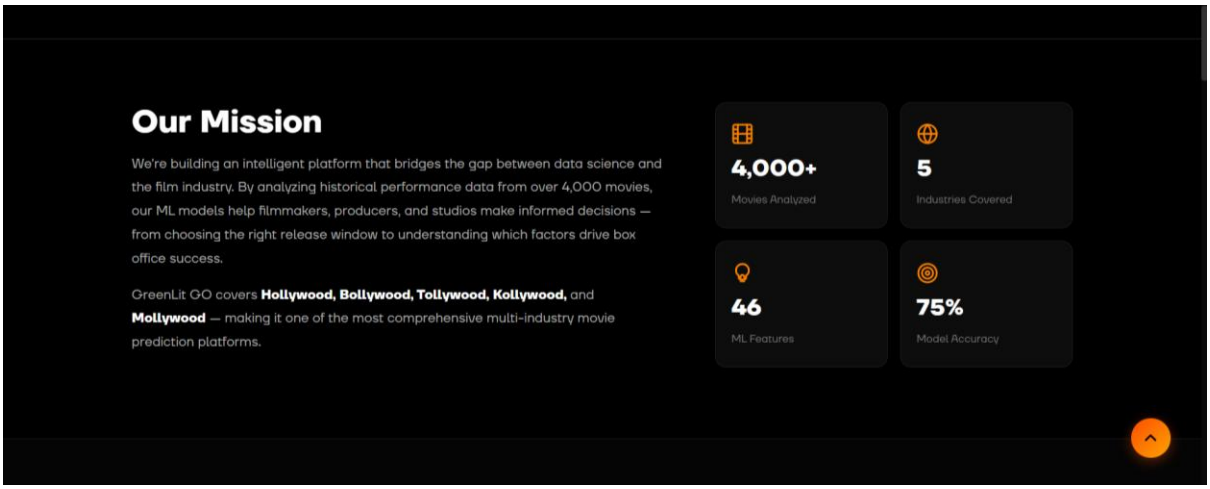

Figure 4.22 About Us Page - About Info

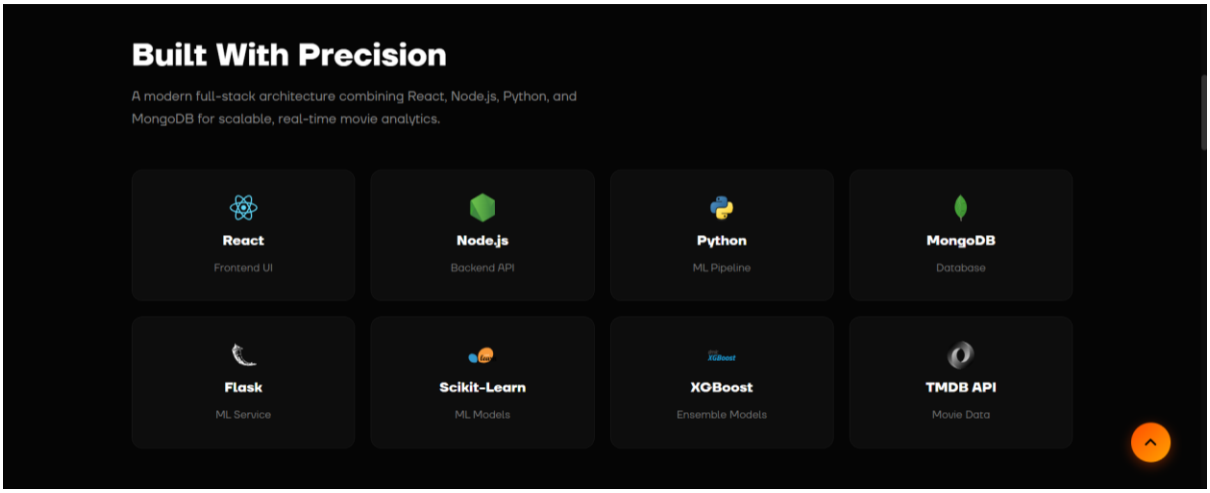

Figure 4.23 About Us Page - Technology Used

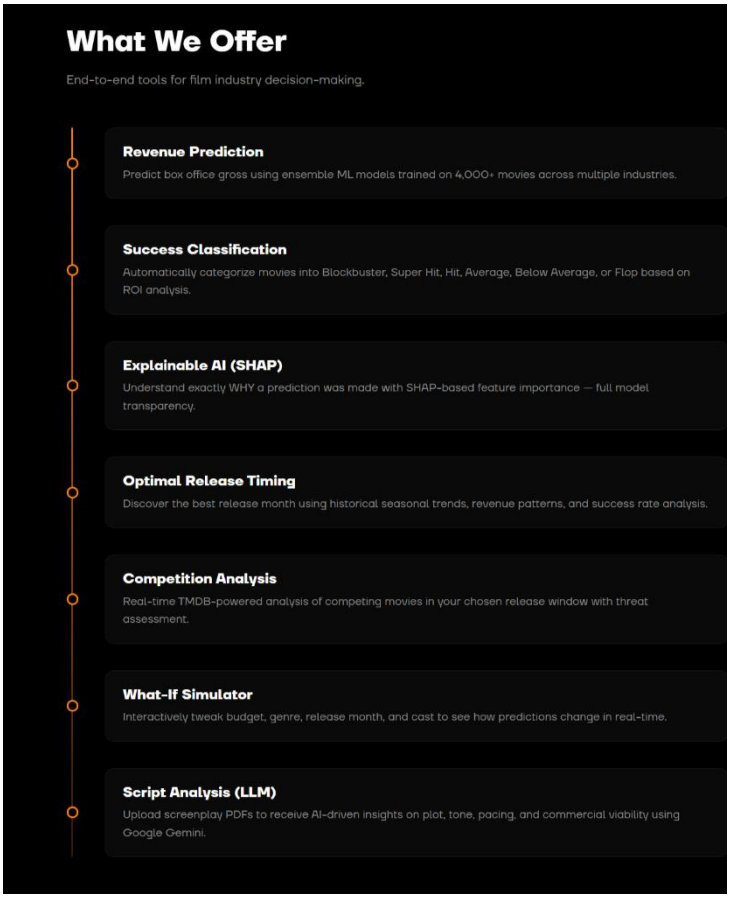

Figure 4.24 About Us Page - What We Offer

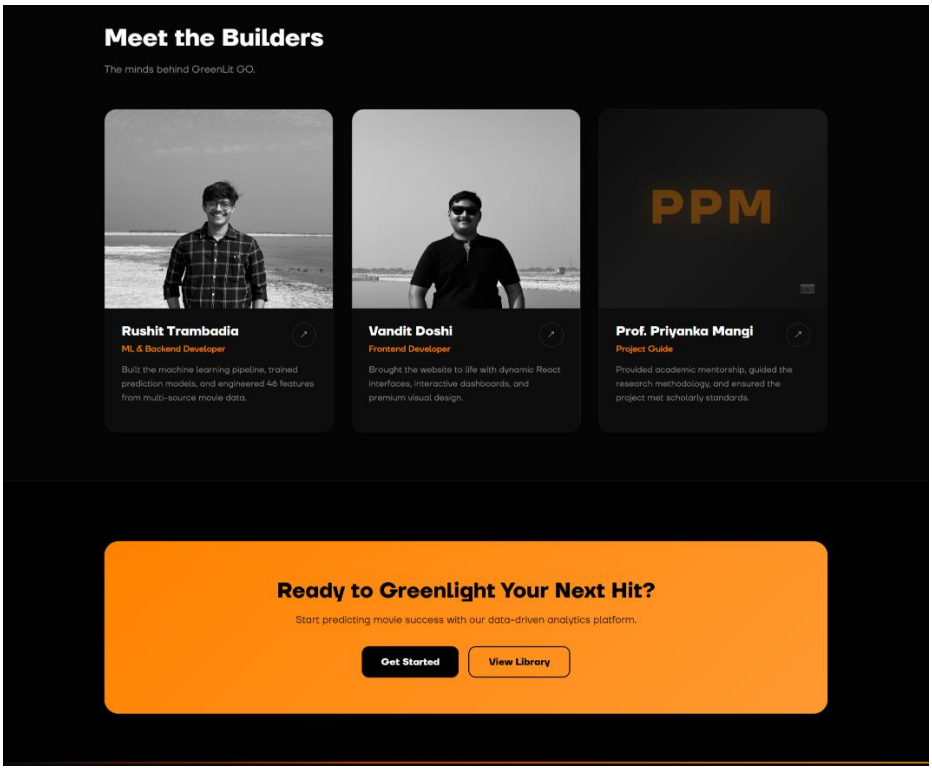

Figure 4.25 About Us Page - Builders and CTA Section

4.6 Testing Plan / Strategy

Testing was structured into three levels:

1. Unit Testing: Individual functions and methods tested in isolation. Feature engineering functions, prediction logic, imputation routines, and API endpoint handlers were each tested with known inputs and expected outputs.
2. Integration Testing: End-to-end request flows tested across component boundaries. A prediction submitted from the React frontend was traced through the Express server, into the Flask ML service, and back to verify the complete pipeline.
3. System Testing: Full user scenarios tested from the browser. Browsing movies, applying filters, submitting predictions, and viewing results were all tested to ensure the application behaves correctly as a whole.
4. ML Validation Testing: Model predictions for well-known released movies were compared against actual outcomes to sanity-check that the model's outputs are in the right ballpark.

.

4.7 Test Results and Analysis

4.7.1 Test Cases

Feature Engineering Test Cases

Table 4.8 Feature Engineering Test Cases

Test ID	Test Condition	Expected Output	Actual Output	Remark
FE-01	Budget $100M, year 2000, inflation normalize	budget_adjusted ≈ $197M	$196.7M	PASS — 3% annual compound applied correctly
FE-02	Release month = 12, cyclic encode	sin ≈ 0.0, cos ≈ 1.0	sin=0.0, cos=1.0	PASS

FE-03	Release month = 6, cyclic encode	sin ≈ 0.0, cos ≈ -1.0	sin=0.0, cos=-1.0	PASS — June correctly opposite to December
FE-04	Genres = ["Action", "Adventure"]	genre_action=1, genre_adventure=1, rest=0	All correct	PASS
FE-05	Cast popularity [45.2, 12.8, 8.1]	avg=22.03, max=45.2	avg=22.03, max=45.2	PASS
FE-06	Budget=$200M, total_talent=0	No division-by-zero crash	Clipped to 0.1, value=2B	PASS — edge case handled
FE-07	Missing runtime field	Defaults to 120 minutes	runtime=120	PASS

Prediction Logic Test Cases

Table 4.9 Prediction Logic Test Cases

Test ID	**Test Condition**	**Expected Output**	**Actual Output**	**Remark**
PL-01	All empty inputs (demo mode)	Returns demo prediction payload	Returned defaults	PASS
PL-02	$200M Hollywood action sequel	Revenue between $200M-$2.4B	$482M	PASS — plausible range
PL-03	$5M Mollywood drama	Revenue under $50M	$8.2M	PASS — Mollywood scale applied
PL-04	Negative ROI scenario	Category = Average or Flop	Category = Flop	PASS

PL-05	ROI > 300%	Category = Blockbuster	Category = Blockbuster	PASS — override triggered
PL-06	Predicted rating bounds	Between 3.0 and 9.5	7.4	PASS — clamped correctly

API Endpoint Test Cases

Table 4.10 API Endpoint Test Cases

Test ID	Test Condition	Expected Output	Actual Output	Remark
API-01	POST /api/predict with valid payload	200 status + predictions	Received predictions JSON	PASS
API-02	POST /api/predict missing budget	400 error with message	"Missing required fields"	PASS
API-03	GET /api/movies?industry=bollywood	Only Bollywood movies	Verified all results	PASS
API-04	GET /api/movies/search?q=Avengers	Matching movies returned	4 results	PASS
API-05	POST /api/explain with valid data	Feature importance array	7 features returned	PASS

Integration Test Cases

Table 4.11 Integration Test Cases

Test ID	Test Condition	Expected Output	Actual Output	Remark
INT-01	Browse Released Movies page	Movie cards with posters load	Cards rendered correctly	PASS
INT-02	Submit prediction via UI form	Results displayed in prediction cards	Animated cards appeared	PASS
INT-03	Script analysis submission	Structured scores returned	Commercial potential score displayed	PASS
INT-04	ML Service cold start	Models load without errors	Loaded in ~3 seconds	PASS
INT-05	Submit $5B budget (extreme value)	System applies ceiling logic	Capped and processed	PASS
INT-06	TMDB API fetch on non-VI ISP (e.g., Jio, Airtel)	API request fails/times out	Request failed (CORS/DNS block)	FAIL — ISP level blocking (Resolved via VPN/Proxy)

System Test Cases

Table 4.12 System Test Cases

Test ID	Test Condition	Expected Output	Actual Output	Remark
SYS-01	Full prediction end-to-end	Revenue + category + features shown	All displayed correctly	PASS
SYS-02	Filter by industry + genre + year	Only matching movies shown	Correct filtering	PASS
SYS-03	All Insights charts render	15+ chart types loaded	All rendered with data	PASS
SYS-04	Talent search for "Christopher Nolan"	Director popup with photo and popularity	Displayed correctly	PASS
SYS-05	Mobile layout (375px width)	Responsive stacking	Navigation collapsed, cards stacked	PASS
SYS-06	ML service stopped + prediction attempted	Clear error message shown	"ML service unavailable"	PASS

ML Validation Test Cases (Known Movies)

Table 2.13 ML Validation Tests

Test ID	Movie	Actual Category	Predicted Category	Actual Revenue	Predicted Revenue	Remark
ML-01	Project Hail Mary	Blockbuster	Blockbuster	$400M	$581M	Off by one category

ML-02	Wuthering Heights	Super Hit	Super Hit	$230M	$264M	Directionally Correct
ML-03	Bhoot Bangla	Not Yet Released	Average	Not Yet Released	$38M	To be release

CHAPTER 5

CONCLUSION AND FUTURE ENHANCEMENTS

5.1 Overall Analysis of Project Viabilities

Looking back at where we started and where GreenLit Go ended up, the project has proven viable on all three fronts — technical, economic, and operational.

Technical viability was the area with the most uncertainty going in. The core question was whether a machine learning model could predict box office revenue with any meaningful accuracy using only pre-release information. After extensive feature engineering, hyperparameter tuning, and ensembling, the answer turned out to be yes — with an R^2 of 0.75, the model captures roughly three-quarters of the variance in revenue. This is not perfect, but it is far above what we initially expected, especially considering the model must handle genuinely unreleased movies where several key features need to be intelligently estimated from pre-release signals rather than observed directly.

Economic viability was never in doubt. The entire project was built using free tools. MongoDB Atlas free tier, open-source ML libraries, free API quotas from TMDB, OMDB, YouTube, and Google Gemini — the total out-of-pocket cost was effectively zero. There was no hardware investment either, since development ran comfortably on existing personal laptops.

Operational viability is solid for a local deployment. The system requires three terminal commands to start (one for each service), and a user with no technical background can use the prediction sandbox and movie browser without any training. Deploying to a cloud platform like Render or Railway would require minor configuration changes but no architectural rework.

5.2 Overall Analysis of Project Viabilities

Problem 1: Industry Revenue Scale Mismatch

Encountered: The model was trained on predominantly Hollywood data (72% of the dataset). When used for Bollywood or South Indian movies, it consistently predicted unrealistically high revenues because it had learned Hollywood-scale revenue patterns.

Solution: We implemented industry-specific scaling factors that adjust the raw prediction output based on the target industry's typical revenue distribution (Bollywood at 0.25x, Tollywood at 0.18x, Kollywood at 0.15x, Mollywood at 0.08x of Hollywood scale).

Problem 2: OpenMP Threading Conflicts on Windows

Encountered: When running the ML service on Windows, the XGBoost and LightGBM libraries caused an OpenMP threading error that would hang the Flask process indefinitely.

Solution: Setting OMP_NUM_THREADS=1 and KMP_DUPLICATE_LIB_OK=TRUE as environment variables before starting the ML service resolved the conflict.

Problem 3: Imputing Features for Unreleased Movies

Encountered: The model expects 46 features, but upcoming movies do not have actual values for vote_average, vote_count, popularity, opening_gross, etc. Setting these to zero would bias every prediction toward "obscure/unknown film."

Solution: We built a smart imputation system that estimates these missing values from correlated pre-release signals — budget tier, talent popularity, sequel status, and trailer engagement — calibrated against the training data distribution.

Problem 4: Predicted Rating Calculation

Encountered: Initially, the predicted audience rating was calculated incorrectly as confidence / 10, which produced ratings clustering around 7.0-8.0 regardless of the movie's actual profile.

Solution: We rewrote the rating calculation to use the imputed vote_average as a base, then adjust it up or down based on the predicted success category and ROI, with hard clamping between 3.0 and 9.5.

Problem 5: ISP/DNS Blocking of TMDB API

Encountered: During development and testing of the "Upcoming Movies" page, we noticed that features relying on the TMDB API—such as the Director/Cast auto-complete search and the optimal release windows—would randomly fail or timeout. Upon testing, we determined these features only worked when connected to a VI (Vodafone Idea) network or hotspot, but failed on others (e.g., Jio, Airtel). This happens because some Indian ISPs at the DNS routing level actively block TMDB domains.

Solution: The immediate workaround during the project testing was to ensure the device was connected to a compatible network (VI Hotspot) or use a VPN. The permanent, production-ready solution to bypass client-side ISP blocking is to proxy all TMDB API requests through our own Node.js backend. By moving the TMDB fetch() requests from the React frontend to the backend server, the client only needs to communicate with our unblocked API routes, completely bypassing local ISP restrictions on tmdb.org.

5.3 Summary of Project Work

GreenLit Go was developed over five Agile sprints spanning approximately three months. The work can be summarized in five phases:

1. Data Foundation (Sprint 1): Collected 4,000+ movie records from TMDB, OMDB, YouTube, and Box Office Mojo. Set up MongoDB Atlas and designed the document schema.
2. ML Pipeline (Sprint 2): Built the feature engineering pipeline (46 features), trained initial models, and established baseline performance metrics.
3. Application Development (Sprint 3): Developed the React frontend with seven pages, the Express backend with RESTful APIs, and integrated the Flask ML service into the prediction workflow.

4. Quality Audit (Sprint 4): Identified data leakage risk and built the imputation pipeline. Re-engineered the prediction workflow. Added Optuna-based hyperparameter tuning. Rebuilt the stacking ensemble from scratch.
5. Polish and Delivery (Sprint 5): Added industry-aware classification, smart imputation for upcoming movies, AI script analysis via Gemini, per-prediction feature importance, and comprehensive UI polish with glassmorphic design.

5.4 Limitations

Dataset Imbalance: Approximately 72% of the training data comes from Hollywood. Predictions for Bollywood, Tollywood, Kollywood, and Mollywood are less reliable due to limited training samples for these industries.

No Marketing Spend Data: Marketing and advertising budgets are a significant driver of box office performance but are almost never publicly disclosed. The model cannot account for this factor.

Temporal Drift: Movie economics change over time — streaming disruption, pandemic effects, shifting viewer habits. The model may gradually become less accurate if not retrained with recent data.

Five-Class Granularity: The five success categories (Blockbuster through Flop) mean that near-miss misclassifications are common. A movie predicted as "Hit" that was actually "Super Hit" is technically wrong but not meaningfully so. A binary classification would score higher but provide less useful information.

No Real-Time Social Sentiment: The system does not monitor live social media conversations. Pre-release buzz on Twitter/X, Reddit, or Instagram could be a powerful signal but is not currently captured.

No User Accounts: The system does not support user registration, login, or personalized dashboards. All users see the same interface.

5.5 Future Enhancement

Social Media Sentiment Integration

Adding NLP-based sentiment analysis on pre-release social media posts (Twitter/X, Reddit, Instagram) would give the model a direct measure of audience excitement or skepticism. A fine-tuned BERT or DistilBERT model could classify posts about upcoming movies as positive, neutral, or negative, creating a sentiment score feature.

User Authentication

Login system for saved predictions and history

Prediction History

Track and compare past predictions vs actual results

Automated Retraining Pipeline

Using Apache Airflow or a simple scheduled job to retrain the model monthly as new movies are added to the database would keep the model's understanding of market dynamics current without manual intervention.

Geographic Revenue Decomposition

Predicting domestic vs. international vs. China revenue splits separately would help distribution teams make territory-specific release and marketing decisions.

Advanced Script Analysis with Fine-Tuned LLMs

Moving beyond the current Gemini-based analysis to fine-tuned models that can assess full screenplay structure — three-act pacing, character arc depth, dialogue quality — and correlate these with commercial outcomes.

Streaming Revenue Tracking

As more movies skip theatrical release or have shortened theatrical windows, incorporating streaming viewership data from Netflix Top 10, Disney+ reports, and Amazon Prime Video would provide a fuller picture of commercial performance.

Collaborative Feedback Loop

Adding user accounts with the ability to rate predictions and provide outcome feedback would enable the system to learn from its own successes and mistakes over time.

5.6 Project Outcomes and Discussion

The project successfully delivered on all its stated objectives:

Table 5.1 Project Outcomes

Objective	**Outcome**
Aggregate data from 4+ sources	TMDB, OMDB, YouTube, BOM — 4,000+ movies
Engineer 40+ pre-release features	46 features including inflation normalization, cyclic encoding, talent metrics
Leakage-aware pipeline	Post-release features identified; imputed from pre-release signals for upcoming movies
Stacking ensemble with Optuna	RF + XGB + LGBM + KNN under Ridge meta-learner
$R^2 > 0.70$ on clean data	Achieved $R^2 = 0.75$
Per-prediction explainability	Feature importance personalized to each movie's profile
Full-stack web application	React + Node.js + Flask + MongoDB Atlas
Multi-industry support	5 industries with calibrated thresholds
AI script analysis	Gemini API for commercial potential scoring

The most meaningful outcome of this project, from a technical standpoint, is the data leakage audit. It taught us that a model's headline accuracy number is meaningless without understanding how it would perform on genuinely new data. An R^2 of 0.93 from early experiments that leaned heavily on opening weekend gross is far inferior — for actual forecasting purposes — to the final R^2 of 0.75, where the pipeline has been rebalanced with a smart imputation system that handles missing post-release data for upcoming films. Most

published work in this space does not even acknowledge this distinction, let alone address it architecturally.

From a practical standpoint, GreenLit Go demonstrates that machine learning can add value to the movie greenlighting process without requiring massive infrastructure or proprietary datasets. Everything in this project was built using freely available tools and publicly accessible data. A small production house or an independent filmmaker can run this system on a regular laptop and get a quantitative perspective before committing to a project.

References

[1] Cinelytic Inc., "AI-Driven Content Intelligence Platform for Film Industry," cinelytic.com, 2024. [Online]. Available: https://www.cinelytic.com

[2] ScriptBook NV, "AI Screenplay Analysis and Box Office Prediction," scriptbook.io, 2024. [Online]. Available: https://www.scriptbook.io

[3] Vault AI, "Predictive Content Intelligence Platform for Media and Entertainment," vault-ai.com, 2024. [Online]. Available: https://www.vault-ai.com

[4] The Numbers, "Box Office Data, Movie Financial Analysis and Film Industry Economics," the-numbers.com, 2024. [Online]. Available: https://www.the-numbers.com

[5] V. Udandarao and P. Gupta, "Movie Revenue Prediction using Machine Learning Models," arXiv preprint arXiv:2405.11651, May 2024. [Online]. Available: https://arxiv.org/abs/2405.11651

[6] J. Xu, "Long-Range Movie Box Office Prediction Based on Machine Learning," Highlights in Science, Engineering and Technology, vol. 92, pp. 308–315, 2024, doi: 10.54097/wzq9md55.

[7] S. Tang, "The Box Office Prediction Model Based on the Optimized XGBoost Algorithm in the Context of Film Marketing and Distribution," PLoS ONE, vol. 19, no. 10, Oct. 2024, doi: 10.1371/journal.pone.0309227.

[8] Q. Chao, E. Kim, and B. Li, "Movie Box Office Prediction With Self-Supervised and Visually Grounded Pretraining," in Proc. IEEE Int. Conf. Multimedia and Expo (ICME), 2023.

[9] Q. I. Mahmud et al., "A Machine Learning Approach to Predict Movie Revenue Based on Pre-Released Movie Metadata," J. Comput. Sci., vol. 16, no. 6, pp. 749–767, 2020, doi: 10.3844/jcssp.2020.749.767.

[10] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, San Francisco, CA, USA, 2016, pp. 785–794.

[11] G. Ke et al., "LightGBM: A Highly Efficient Gradient Boosting Decision Tree," in Proc. 31st Int. Conf. Neural Information Processing Systems (NeurIPS), Long Beach, CA, USA, 2017, pp. 3146–3154.

[12] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A Next-generation Hyperparameter Optimization Framework," in Proc. 25th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, Anchorage, AK, USA, 2019, pp. 2623–2631.

[13] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[14] The Movie Database (TMDB), "TMDB API Documentation," themoviedb.org, 2024. [Online]. Available: https://developer.themoviedb.org

[15] Open Movie Database, "OMDB API Documentation," omdbapi.com, 2024. [Online]. Available: https://www.omdbapi.com

[16] Google LLC, "YouTube Data API v3 Documentation," developers.google.com, 2024. [Online]. Available: https://developers.google.com/youtube/v3

Regular Report Diary (Original Copy)

Consent Letter

Invention Disclosure Form

Project Outcome Documents

Research Paper Draft, Website/Application deployment/publishing details, Start Up related official documents etc. (all whichever are relevant to achievement of Project Outcomes)